Data Structure and Algorithms

Lecture 1: Introduction

Understanding the organization, management, and storage of data for efficient access and modification.

Dr. Amthal Kh. Mousa **Lect. Zainab M. Fadhil**

What is a Data Structure?

Data structure is a complex data type with two equally important parts:

1. Representation

A specific representation or way of organizing a collection of more than one element (an individual value or data point).

2. Functionality

A specific functionality or operations such as adding, retrieving, and removing elements.

Abstract Data Types (ADT)

Although values for complex data are diverse, computer scientists design data structures to be reused for other problems.

> **Definition**
>
> An **Abstract Data Type (ADT)** consists of all data structures that share common functionality but differ in specific representation.

Computer scientists categorize data structures according to their functionality without considering their specific representation.

Advantages of Data Structure

- **Organization:** Data structures define a systematic way to organize and store data for easy retrieval and modification.
- **Efficiency:** Choosing the right data structure optimizes operations like searching, sorting, and updating, ensuring efficient use of memory and processing power.
- **Abstraction:** They provide a layer of abstraction that simplifies the representation of data in programming.
- **Other benefits:** Little redundant code, facilitates software reuse, and makes code easier to understand, implement, test, and modify.

Real-World Applications

Arrays:	Representing tabular data, image pixels.
Stacks:	Undo operations in text editors, browser history.
Queues:	Process scheduling in operating systems.
Trees:	File systems, decision trees in AI.
Graphs:	Social networks, shortest path algorithms in maps.

Data Structure Types: Linear

Linear Data Structure: A category where elements are ordered in a line.

1. Array List

A data structure that stores list elements next to each other in memory.

2. Linked List

Does not necessarily store list elements next to each other. Works by maintaining, for each element, a link to the next element in the list.

Both are linear because their elements are organized in a line, one after the other.

Data Structure Types: Non-Linear

Non-Linear Data Structure

Specializes in implementing sets, maps, and priority queues.

Key Characteristic: Elements are organized in a *hierarchy* rather than in a line.

Types of data structures

- Arrays
- Queues
- Stacks
- Linked lists
- Trees
- Graphs
- Hash

Standard Template Library (STL)

The C++ Standard Library comes powered with several data structures. The STL contains many class and function templates used to store, search, and perform algorithms on data structures.

Four Main Components:

1. Containers
2. Iterators
3. Functions
4. Algorithms

STL: Containers

Definition

Containers are generic data structures provided by the STL (such as `vector`, `list`, and `map`) designed to manage and store collections of objects in memory efficiently.

Common Methods on Containers (1/2)

Many C++ container types have methods with similar names.

`container constructor`
A function that initializes an object.

`container.size()`
Return the number of elements in the container.

`container.empty()`
Return true iff the container is empty (has size() == 0).

`container.clear()`
Remove all elements from the container, leaving it empty().

Common Methods on Containers (2/2)

`container[index]`
Return a modifiable reference to the element at position index. Supported on vectors, maps, hash tables (not linked lists).

`container.back()`
Return the last element. Supported on sequences like vectors and linked lists (not maps/hash tables).

`container.push_back(value)`
Add a new element to the end containing a copy of value.

`container.pop_back()`
Remove the last element in the container.

STL: Iterators

The Concept

An iterator indicates a current position in the container; the value at that position can be found by “dereferencing” the iterator as if it were a pointer.

C++ iterators are powerful, efficient, and "their own thing."

Looping Example (Standard):

```
std::vector<int> a = ...;
for (auto it = a.begin(); it != a.end(); ++it)
{
    std::cout<< (*it ) <<" \n";
}
```

Iterators: For-Each Loop

Because iterator methods have similar names for all containers, the exact same loop will work for a linked list or set, and related loops will work for maps and hash tables!

The "For-Each" Loop:

```
for (auto& elt : a)
{
    std::cout<< elt <<" \n";
}
```

Note: The for-each loop compiles into the iterator version behind the scenes.

Iterators have long, complicated type names, so we usually use auto to avoid giving the whole type name: **auto it = a.begin();**)

Important Iterator Methods

`container.begin()`
Return an iterator pointing to the first element in the container.

`container.end()`
Return an iterator pointing "one past the end" of the container. Must never be dereferenced.
If empty: begin() == end()

`++iterator`
Move the iterator to point at the next element. Do not call when it == container.end().

`--iterator`
Move the iterator to point at the previous element. Do not call when it == container.begin().

Standard Algorithms (Sorting)

The `<algorithm>` header defines useful algorithms that work on all kinds of C++ data structures.

`std::sort(first, last)`
Sort the elements of the range. Element objects must be comparable using <.

`std::sort(first, last, compare)`
Sort the elements of the range according to a compare function.
Note: compare(e1, e2) should return true if e1 sorts below e2.

Note: 'first' and 'last' denote iterators. 'last' is not included in the range.

Standard Algorithms (Searching)

`std::find(first, last, value)`

Return an iterator to the first element of the range that equals value, or last if no element is found.

`std::find_if(first, last, predicate)`

Return an iterator to the first element of the range for which predicate(*it) returns true, or last if no element is found.

`std::lower_bound(first, last, value)`

Return an iterator to the first position in the range that is greater than or equal to value.

Requires sorted range. Uses binary search (efficient for vectors/arrays).

Conclusion

Summary

✅ **Data Structures** combine representation and functionality.

✅ **ADTs** allow us to categorize structures by behavior.

✅ **STL** provides powerful, reusable tools: Containers, Iterators, and Algorithms.

✅ **Iterators** bridge the gap between containers and algorithms.

Linked Lists

A linked list is a linear collection of specially designed data elements, called *nodes*, linked to one another by means of pointers. Each *node* is divided into two parts: the first part contains the *information* of the element, and the second part contains the *address* of the next node in the linked list. Address part of the node is also called *linked* or *next* field.

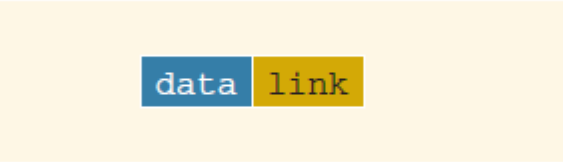


fig1: structure of a node

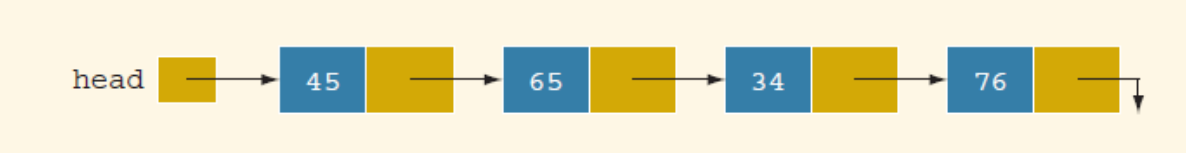


fig2: linked list

Note: the down arrow in the last node indicates that this link field is NULL.

-the next pointer of the last node contains a special value, called the NULL pointer, which does not point to any address of the node, that is NULL pointer indicates the end of the linked list .

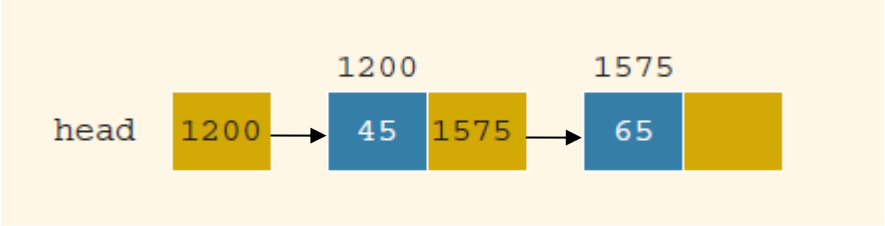


fig3: Linked list and values of the links

Representation of Linked List

The Linked List can be represented in memory in the following declaration (we need to declare each node as **class** or **struct**.

```
struct   node
   {
        int  data ;   //Instead of 'data' we also use 'info'
        node *link ; // Instead of 'Next' we also use 'Link'
   };
typedef  struct node  *node
```

ADVANTAGES AND DISADVANTAGES

Linked list have many *advantages* and some of them are:

1. Linked list are dynamic data structure. That is, they can grow or shrink during the execution of a program.
2. Efficient memory utilization: In linked list (or dynamic) representation, memory is not pre-allocated. Memory is allocated whenever it is required. And it is deallocated (or removed) when it is not needed.
3. Insertion and deletion are easier and efficient. Linked list provides flexibility in inserting a data item at a specified position and deletion of a data item from the given position.
4. Many complex applications can be easily carried out with linked list.

Linked list has following *disadvantages*

1. More memory: to store an integer number, a node with integer data and address field is allocated. That is more memory space is needed.
2. Access to an arbitrary data item is little bit cumbersome and also time consuming.

Linked Lists: some properties

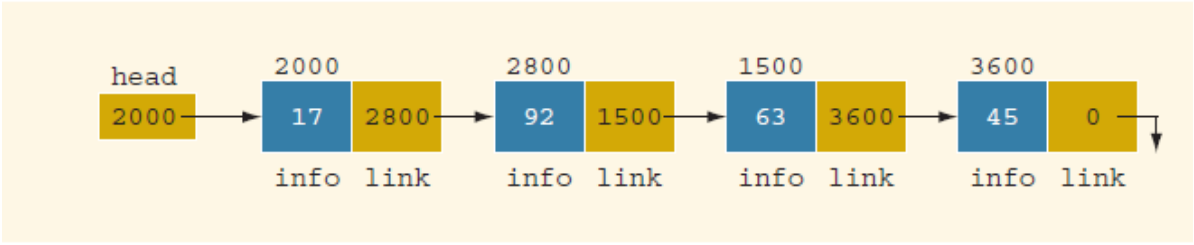


fig 4 : Linked list with four nodes

	Value	Explanation
head	2000	
head->info	17	Because head is 2000 and the info of the node at location 2000 is 17
head->link	2800	
head->link->info	92	Because head->link is 2800 and the info of the node at location 2800 is 92

- Suppose that current is a pointer of the same type as the pointer head, then the statement:

  current = head ;

copies the value of head into current .

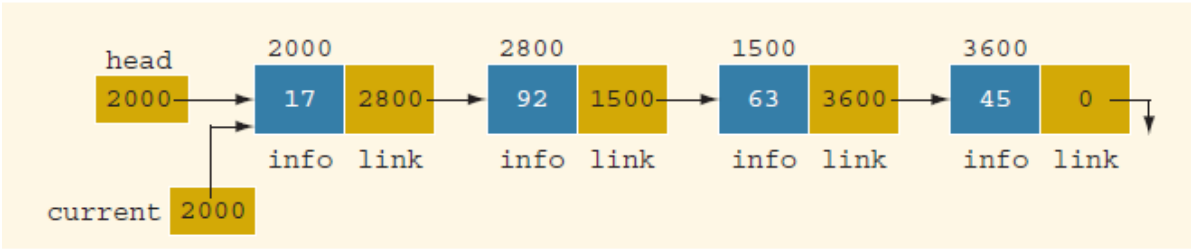


fig 5: linked list after the statement **current = head** ; executes

clearly, in figure 5 :

	Value
Current	2000
Current –> info	17
Current –> link	2800
Current –> link –> info	92

- consider the statement

  current = current –>link ;

  this statement copies the value current –>link ,which is 2800 ,into current.

  Therefore, after this statement executes, current points to the second node in the list.

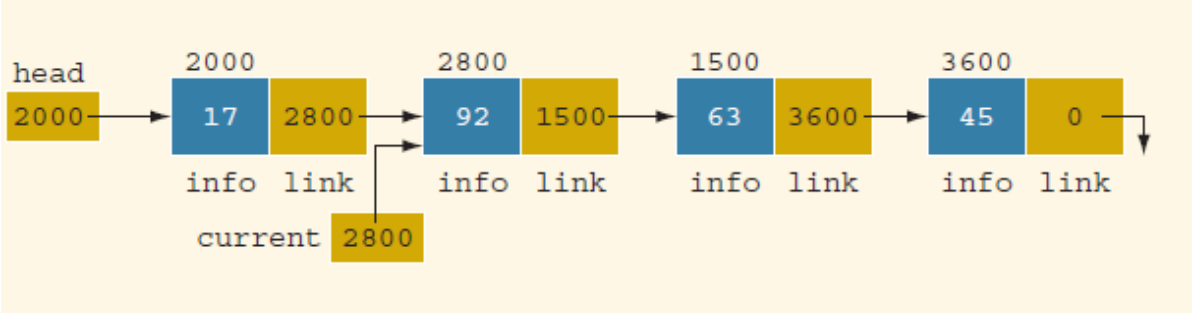


fig 6 : linked list after the statement **current = current –>link** ;executes

in fig 6 :

	Value
Current	2800
Current –> info	92
Current –> link	1500
Current –> link –> info	63
head–> link –>link	1500
head–>link–>link–>info	63
head–>link–>link–>link	3600
head–>link–>link–>link–>info	45
current–> link –>link	3600
current–>link–>link–>info	45
current–>link–>link–>link	0 (that is NULL)
	Does not exist

Operations on Linked List

The basic operations of a linked list are as follows:

search the list to determine whether a particular item in the list, **insert** an item in the list, and **delete** an item from the list. These operations require the list to be **traversed** (is the process of going through all the nodes from one end to another end of a linked list.

Concatenation is the process of appending the second list to the end of the first list.

- We cannot use the pointer head to traverse the list because if we use head to traverse the list, we would lose the nodes of the list.

- We always want **head** to point to the first node, suppose that current is a pointer of the same type. the following code **traverses** the list :

```
current = head ;
while( current != NULL)
{
      // process the current node
 current = current –> link ;
 }
```

Example: the following code outputs the *data* stored in each node:

```
current = head ;
while( current  !=  NULL)
{
  cout<< current–> info << ”   “ ;
   current = current  –> link ;
 }
```

Item insert and delete

Suppose that p points to the node with info 65, and new node with info 50 is to be created and inserted after p .

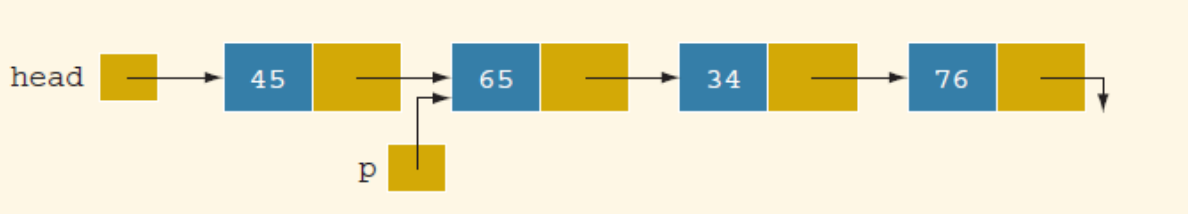


fig 7 : link list before item insertion

```
struct  nodeType
{
     int   info;
    nodeType  * link ;
 };
nodeType   *head , *p , *q ,*newNode;
newNode = new  nodeType ;     // create newNode
newNode  –> info = 50 ;          // store 50 in the new node
```

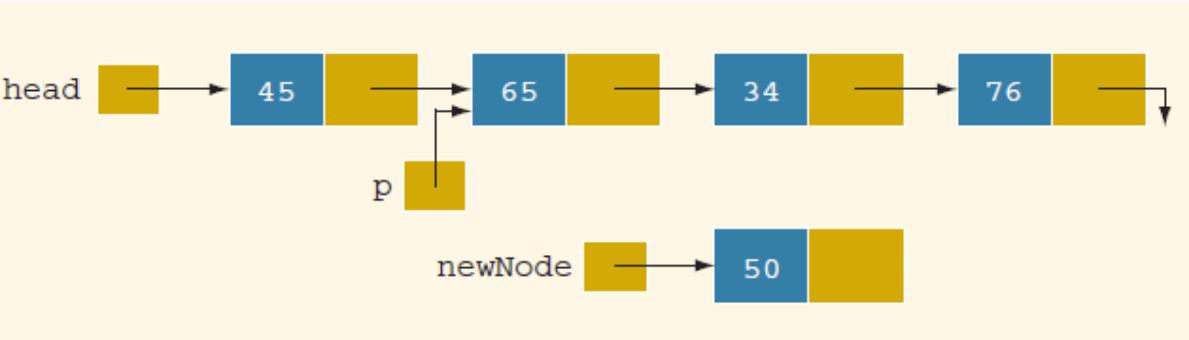


fig 8 : create newNode and store 50 in it

The following statements insert the node in the linked list at the required place:

newNode –>link = p –> link ;

p –> link = newNode ;

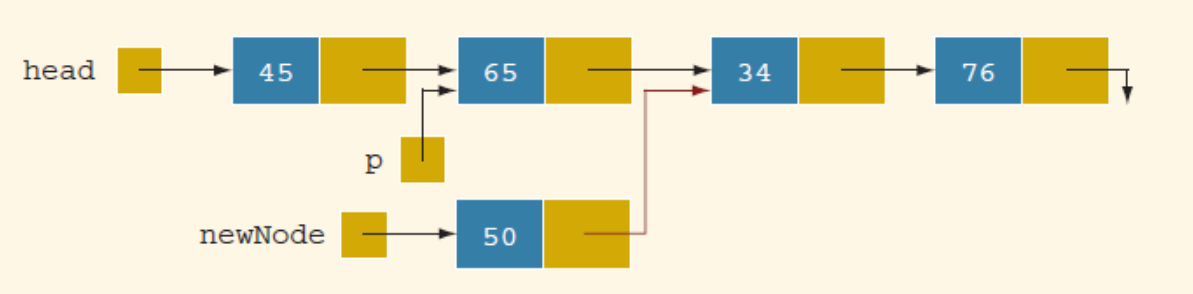


fig 9 :list after the statement **newNode –>link = p –>link** executes ;

After the second statement (that is, p –> link = newNode ;) executes, the resulting list is as show in *figure 10*.

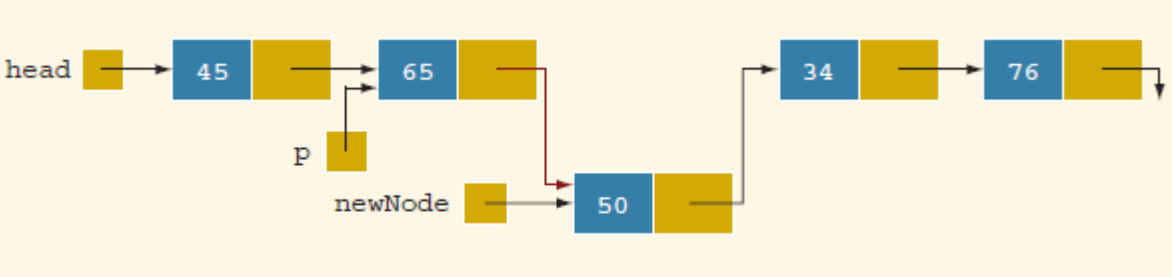


Fig 10: Linked list after the statement **p–> link = newNode** executes

Suppose that we reverse the sequence of the statements and execute the statements in the following order:

p –> link = newNode ;

newNode –> link = p–> link ;

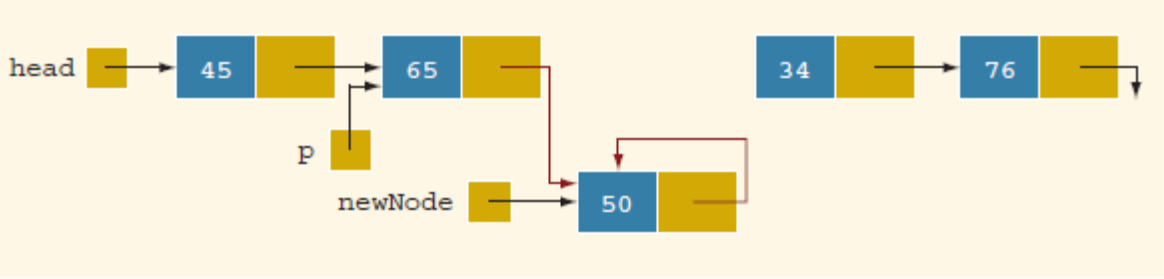


fig 11: Linked list after the execution of the statement p–>link = newNode ;
followed by the execution of the statement newNode –> link = p–>link;

it is clear that newNode points back to itself and the remainder of the list is lost.

Deletion

Consider the Linked list shown in *figure 12*.

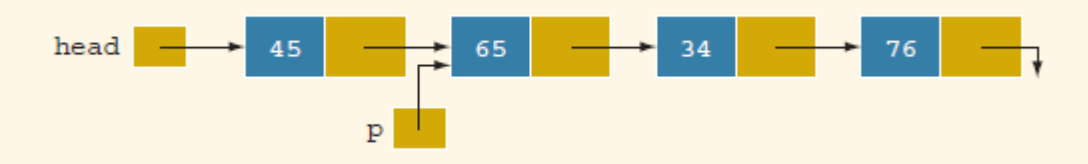


fig 12 : Node to be deleted is with info 34

We need pointer to this node (with info 34).the following statement delete the node from the list and deallocate the memory occupied by this node:

q = p –> link ;

p –> link = q –> link ;

delete q ;

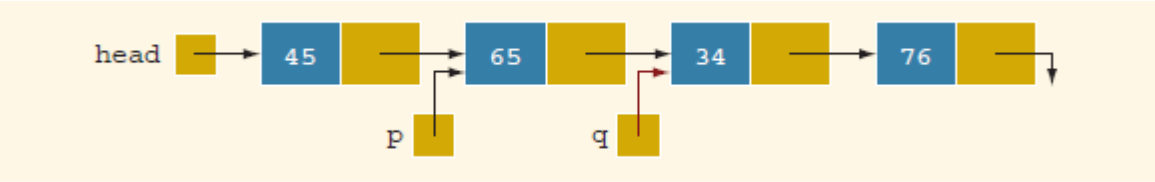


Fig 13: list after the statement q = p –> link

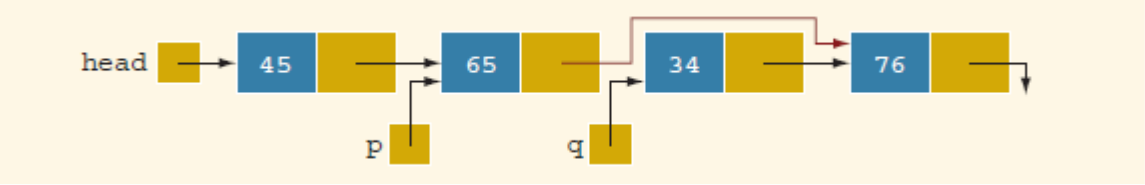


Fig 14 : list after the statement p –> link = q –> link ; executes

Linked List as an ADT

Single Linked list program:

```
#include <iostream>
using namespace std;
class LinkedList
{
  private:
    struct Node
       {
        int data;
        Node* next;
       };

    Node* head;
    int count;

public:
    LinkedList()
       {
        head = nullptr;
        count = 0;
      }

    void insertFirst(int value)
      {
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = head;
        head = newNode;
        count++;
     }

    void deleteFirst()
       {
        if (head != nullptr)
           {
            Node* temp = head;
            head = head->next;
            delete temp;
            count--;
           }
      }
```

```
void insertEnd(int value)
   {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;

    if (head == nullptr)
      {
        head = newNode;
        count++;
        return;
      }

    Node* current = head;
    while (current->next != nullptr)
       {
        current = current->next;
       }
    current->next = newNode;
    count++;
 }
bool insertAfter(int key, int value)
 {
    Node* current = head;
    while (current != nullptr && current->data != key)
       {
        current = current->next;
      }
    if (current == nullptr) return false;

    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = current->next;
    current->next = newNode;
    count++;
    return true;
}
bool insertAt(int pos, int value)
    {
    if (pos < 0 || pos > count) return false;

    if (pos == 0) {
        insertFirst(value);
        return true;
```

```
        }

        Node* prev = head;
        for (int i = 0; i < pos - 1; i++)
            {
            prev = prev->next;
          }
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = prev->next;
        prev->next = newNode;
        count++;
        return true;
    }

    void deleteLast()
       {
        if (head == nullptr)
                return;
        if (head->next == nullptr)
            {
            delete head;
            head = nullptr;
            count--;
            return;
          }
        Node* prev = head;
        while (prev->next->next != nullptr)
            {
            prev = prev->next;
          }
        delete prev->next;
        prev->next = nullptr;
        count--;
    }
    bool deleteNode(int key)
       {
        if (head == nullptr) return false;
        if (head->data == key)
           {
            deleteFirst();
            return true;
          }
        Node* prev = head;
```

```
        while (prev->next != nullptr && prev->next->data != key)
           {
             prev = prev->next;
           }
        if (prev->next == nullptr)
               return false;

        Node* temp = prev->next;
        prev->next = temp->next;
        delete temp;
        count--;
        return true;
    }

    bool deleteAt(int pos)
       {
        if (pos < 0 || pos >= count)
               return false;
        if (pos == 0)
          {
            deleteFirst();
            return true;
          }

        Node* prev = head;
        for (int i = 0; i < pos - 1; i++)
           {
             prev = prev->next;
           }
        Node* temp = prev->next;
        prev->next = temp->next;
        delete temp;
        count--;
        return true;
    }
   void print()
     {
        Node* current = head;
        while (current != nullptr)
            {
            cout << current->data << " ";
            current = current->next;
          }
        cout << "\n";
```

```
     }
    int size() const
       {
        return count;
      }
    ~LinkedList()
       {
        while (head != nullptr)
          {
            deleteFirst();
        }
    }
};
int main()
  {
    LinkedList L;
    L.insertFirst(30);
    L.insertFirst(20);
    L.insertFirst(10);
    cout << "After insertFirst: ";
    L.print();
    L.insertEnd(40);
    cout << "After insertEnd(40): ";
    L.print();
    L.insertAfter(20, 25);
    cout << "After insertAfter(20,25): ";
    L.print();
    L.insertAt(2, 22);
    cout << "After insertAt(2,22): ";
    L.print();
    L.deleteLast();
    cout << "After deleteLast: ";
    L.print();
    L.deleteNode(22);
    cout << "After deleteNode(22): ";
    L.print();
    L.deleteAt(1);
    cout << "After deleteAt(1): ";
    L.print();

    cout << "List size = " << L.size() << "\n";

    return 0;
}
```

Circular linked list

We can, however, change the linear linked list slightly, making the pointer in the **next** member of the last node point back to the first node instead of containing **NULL**. Now our list becomes a **circular linked list** rather than a linear linked list.

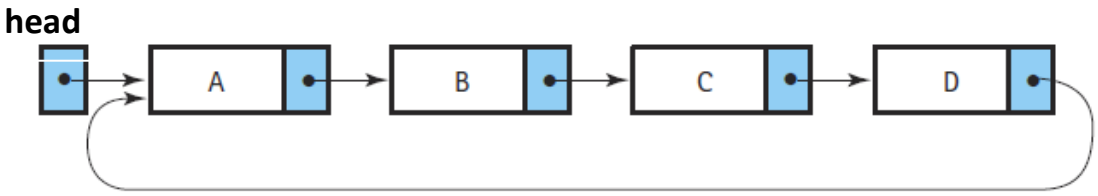


Fig. 15: Representation of circular linked list

In the following algorithms, **listData** represents the pointer to the last node (the tail).

Algorithm 1: Searching a Sorted Circular List

FindItem(listData, target, location, predLoc, found)

- **Input:** `listData` (pointer to tail), `target` (value to find).
- **Output:** `location` (node containing target or its place), `predLoc` (preceding node), `found` (boolean).

1. **Initialize:** * Set `location` ← `listData->next` (Start at Head).
    - Set `predLoc` ← `listData` (Start at Tail).
    - Set `found` ← **false**.
    - Set `moreToSearch` ← **true**.
2. **Search Loop:** While (`moreToSearch` is **true** AND `found` is **false**)
    - **If** `target` < `location->info` **Then**
        - Set `moreToSearch` ← **false** (Target not in list).
    - **Else If** `target` == `location->info` **Then**
        - Set `found` ← **true**.
    - **Else**
        - Set `predLoc` ← `location`.
        - Set `location` ← `location->next`.
        - Set `moreToSearch` ← (`location` != `listData->next`) (Stop if we return to Head).
3. **Return** `location`, `predLoc`, and `found`.

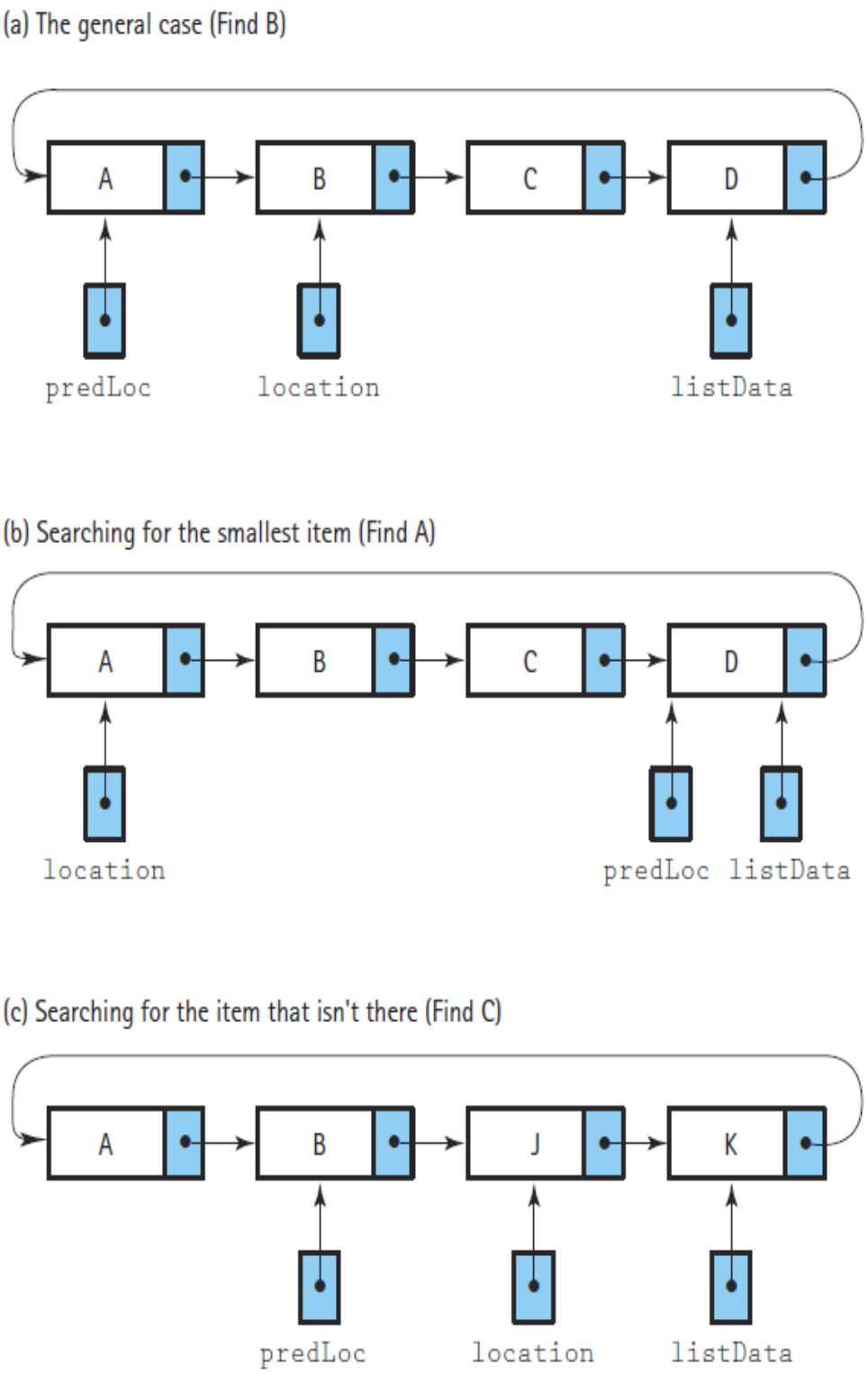


Fig. 16: representation of searching operation

Algorithm 2: Inserting into a Sorted Circular List

InsertItem(listData, item)

- **Input:** `listData` (tail pointer), `item` (data to add).
- **Pre-condition:** List is sorted.

1. **Allocate:** Create `newNode`, set `newNode->info` ← `item`.
2. **Case 1: Empty List**
   - **If** `listData` is **NULL Then**
     - Set `listData` ← `newNode`.
     - Set `newNode->next` ← `newNode`.
3. **Case 2: Non-Empty List**
   - **Search:** Call **FindItem**(`listData`, `item`, `location`, `predLoc`, `found`).
   - **Link:** Set `newNode->next` ← `predLoc->next`.
   - **Re-route:** Set `predLoc->next` ← `newNode`.
   - **Update Tail: If** `listData->info` < `item` **Then**
     - Set `listData` ← `newNode`.
4. **Finalize:** Increment `length`.

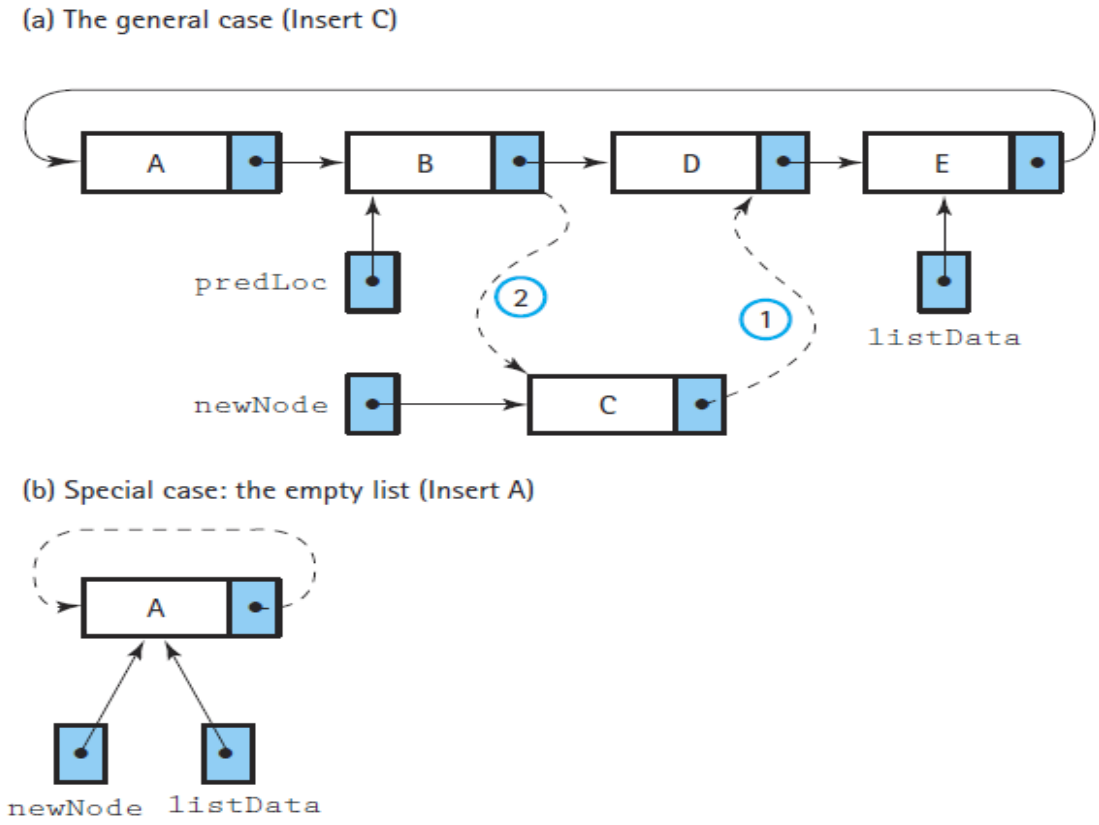


Fig. 17: representation of insert operation

Algorithm 3: Deleting from a Circular List

DeleteItem(listData, item)

- **Input:** `listData` (tail pointer), `item` (value to remove).

1. **Locate:** Call **FindItem**(`listData`, `item`, `location`, `predLoc`, `found`).
2. **Check Found: If** `found` is **false**, **Exit** (Nothing to delete).
3. **Case 1: Single Node List**
   - **If** `predLoc` == `location` **Then**
     - Set `listData` ← **NULL**.
4. **Case 2: Multiple Node List**
   - **Bypass:** Set `predLoc->next` ← `location->next`.
   - **Update Tail: If** `location` == `listData` **Then**
     - Set `listData` ← `predLoc`.
5. **Finalize:** Deallocate `location`, decrement `length`.

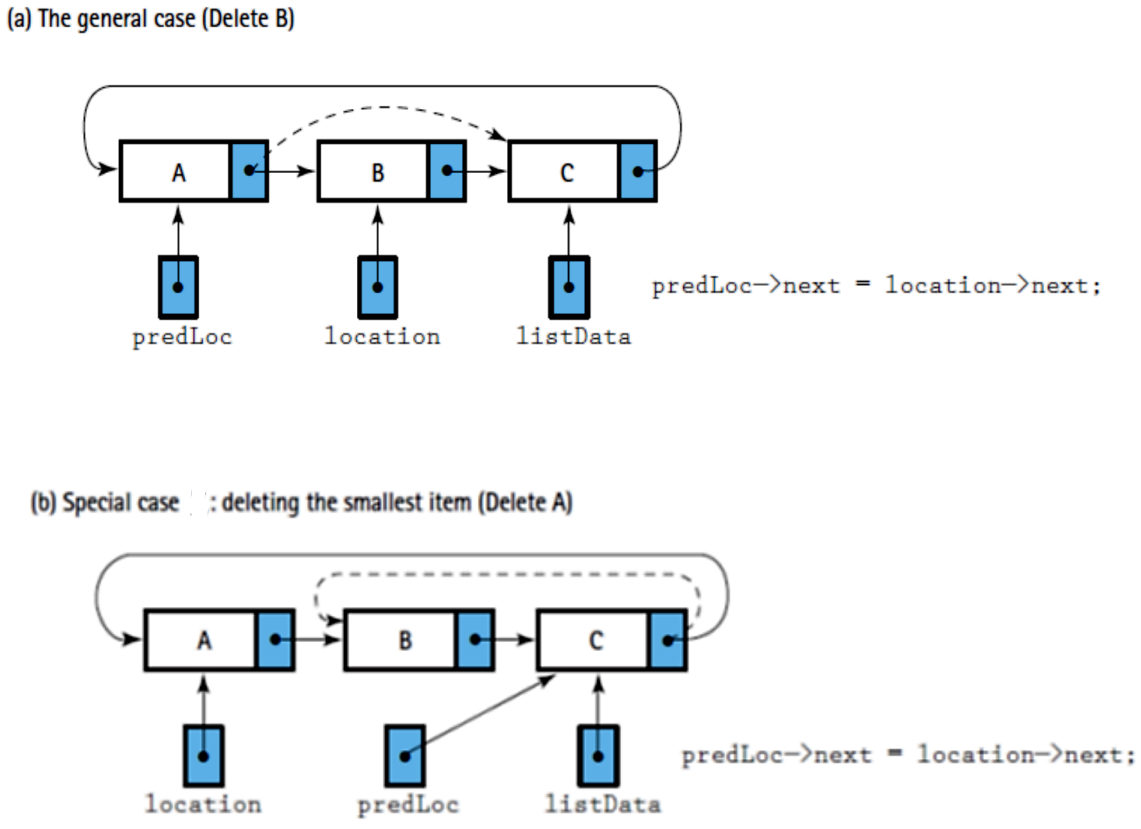


Fig. 18: representation of delete operation

Implementation Using STL (std::list)

std::list is a sequential container that's an abstraction of a *doubly linked list*. Each node contains a pointer to both the next and the previous node. Figure 1 illustrates the logical structure of a std::list. std::lists support fast constant-time insertions and removals using either container end. Mid- container insertions and removals are also performed in constant time since these operations are achievable using pointer manipulations instead of object copies or moves.

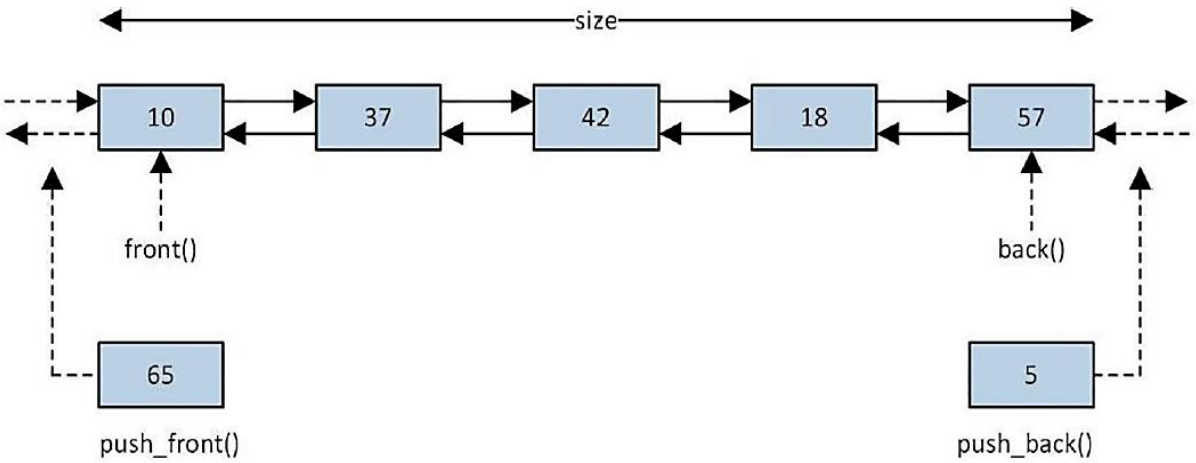


Fig. 19: *Logical structure of std::list*

std::list container does not support random access operations, which means a program can't access its elements using *operator[]* or *at()*. A program must exploit iterators to traverse a *std::list* container, but this movement can be carried out in either direction. Lack of random access operations also means that some STL algorithms can't be used with *std::lists*; however, class *std::list* defines suitable alternatives for some of these algorithms.

Program of Double Linked list using STL

```
#include <iostream>
#include <list>
#include <iterator>
using namespace std;

// Print function (same style as manual version)
void printList(const list<int>& L)
  {
    if (L.empty())
      {
        cout << "(empty)\n";
        return;
      }

    for (auto it = L.begin(); it != L.end(); ++it)
     {
        cout << *it;
        auto nextIt = it;
        ++nextIt;
        if (nextIt != L.end())
            cout << " -> ";
     }
    cout << " -> nullptr\n";
}

// Insert after key
bool insertAfter(list<int>& L, int key, int value)
{
    for (auto it = L.begin(); it != L.end(); ++it)
     {
        if (*it == key)
          {
            ++it;
            L.insert(it, value);
            return true;
          }
    }
    return false;
```

```
}
// Insert at position (0-based)
bool insertAt(list<int>& L, int pos, int value)
{
    if (pos < 0 || pos > (int)L.size())
        return false;

    auto it = L.begin();
    advance(it, pos);
    L.insert(it, value);
    return true;
}

// Delete by key
bool deleteNode(list<int>& L, int key)
{
    for (auto it = L.begin(); it != L.end(); ++it)
     {
        if (*it == key)
          {
            L.erase(it);
            return true;
          }
    }
    return false;
}

// Delete at position
bool deleteAt(list<int>& L, int pos)
{
    if (pos < 0 || pos >= (int)L.size())
        return false;

    auto it = L.begin();
    advance(it, pos);
    L.erase(it);
    return true;
}
```

```
int main()
{    list<int> L;

    // insertFirst
    L.push_front(30);
    L.push_front(20);
    L.push_front(10);
    cout << "After insertFirst: ";
    printList(L);

    // insertEnd
    L.push_back(40);
    cout << "After insertEnd(40): ";
    printList(L);

    // insertAfter
    insertAfter(L, 20, 25);
    cout << "After insertAfter(20,25): ";
    printList(L);

    // insertAt
    insertAt(L, 2, 22);
    cout << "After insertAt(2,22): ";
    printList(L);

    // deleteLast
    L.pop_back();
    cout << "After deleteLast: ";
    printList(L);

    // deleteNode
    deleteNode(L, 22);
    cout << "After deleteNode(22): ";
    printList(L);

    // deleteAt
    deleteAt(L, 1);
    cout << "After deleteAt(1): ";
    printList(L);

    cout << "List size = " << L.size() << endl;
    return 0;
}
```

Singly Link List

```
//Single Linked list program :
#include <iostream>
using namespace std;
class LinkedList {
private:
struct Node {
int data;
Node* next;
};
Node* head;
int count;
public:
LinkedList() {
head = nullptr;
count = 0;
}
void insertFirst(int value) {
Node* newNode = new Node;
newNode->data = value;
newNode->next = head;
head = newNode;
count++;
}
void deleteFirst() {
if (head != nullptr) {
Node* temp = head;
head = head->next;
delete temp;
count--;
}
}
void insertEnd(int value) {
Node* newNode = new Node;
newNode->data = value;
newNode->next = nullptr;
if (head == nullptr) {
head = newNode;
count++;
return;
}
Node* current = head;
while (current->next != nullptr) {
current = current->next;
```

```
}
current->next = newNode;
count++;
}
bool insertAfter(int key, int value) {
Node* current = head;
while (current != nullptr && current->data != key) {
current = current->next;
} if (
current == nullptr) return false;
Node* newNode = new Node;
newNode->data = value;
newNode->next = current->next;
current->next = newNode;
count++;
return true;
}
bool insertAt(int pos, int value) {
if (pos < 0 || pos > count) return false;
if (pos == 0) {
insertFirst(value);
return true;
}
Node* prev = head;
for (int i = 0; i < pos - 1; i++) {
prev = prev->next;
}
Node* newNode = new Node;
newNode->data = value;
newNode->next = prev->next;
prev->next = newNode;
count++;
return true;
}
void deleteLast() {
if (head == nullptr) return;
if (head->next == nullptr) {
delete head;
head = nullptr;
count--;
return;
}
Node* prev = head;
while (prev->next->next != nullptr) {
```

```
prev = prev->next;
}
delete prev->next;
prev->next = nullptr;
count--;
}
bool deleteNode(int key) {
if (head == nullptr) return false;
if (head->data == key) {
deleteFirst();
return true;
}
Node* prev = head;
while (prev->next != nullptr && prev->next->data != key) {
prev = prev->next;
} if (
prev->next == nullptr) return false;
Node* temp = prev->next;
prev->next = temp->next;
delete temp;
count--;
return true;
}
bool deleteAt(int pos) {
if (pos < 0 || pos >= count) return false;
if (pos == 0) {
deleteFirst();
return true;
}
Node* prev = head;
for (int i = 0; i < pos - 1; i++) {
prev = prev->next;
}
Node* temp = prev->next;
prev->next = temp->next;
delete temp;
count--;
return true;
}
void print() {
Node* current = head;
while (current != nullptr) {
cout << current->data << " ";
current = current->next;
```

```
}
cout << "\n";
}
int size() const {
return count;
}
~LinkedList() {
while (head != nullptr) {
deleteFirst();
}
}
};
int main() {
LinkedList L;
L.insertFirst(30);
L.insertFirst(20);
L.insertFirst(10);
cout << "After insertFirst: ";
L.print();
L.insertEnd(40);
cout << "After insertEnd(40): ";
L.print();
L.insertAfter(20, 25);
cout << "After insertAfter(20,25): ";
L.print();
L.insertAt(2, 22);
cout << "After insertAt(2,22): ";
L.print();
L.deleteLast();
cout << "After deleteLast: ";
L.print();
L.deleteNode(22);
cout << "After deleteNode(22): ";
L.print();
L.deleteAt(1);
cout << "After deleteAt(1): ";
L.print();
cout << "List size = " << L.size() << "\n";

system("pause");
return 0;
}
```

Output:

```
After insertFirst: 10 20 30
After insertEnd(40): 10 20 30 40
After insertAfter(20,25): 10 20 25 30 40
After insertAt(2,22): 10 20 22 25 30 40
After deleteLast: 10 20 22 25 30
After deleteNode(22): 10 20 25 30
After deleteAt(1): 10 25 30
List size = 3
```

STL Containers and Operations: Circular Linked List & Doubly Linked List in C++

Circular Linked List Using STL (std::list)

Container: CircularList (using std::list<int>). A circular linked list is implemented using std::list<int>, with custom iterator logic to simulate circular behavior.

Operation	STL Method Used	Description
Insert at End	clist.push_back(val);	Adds a value to the end of the list.
Insert at Front	clist.push_front(val);	Adds a value at the front of the list.
Remove Element	clist.remove(val);	Removes the first occurrence of a value.
Display List	Custom `display()` Method	Iterates through the list to print elements.
Circular Display	Custom `circularDisplay()`	Loops through elements in a circular manner.
Check if Empty	clist.empty();	Checks if the list is empty.
Clear List	clist.clear();	Removes all elements from the list.

Doubly Linked List Using STL (std::list)

Container: DoublyList (using std::list<int>). A doubly linked list is natively supported in STL via std::list<int>.

Operation	STL Method Used	Description
Insert at End	dlist.push_back(val);	Adds a value to the end of the list.
Insert at Front	dlist.push_front(val);	Adds a value at the front of the list.
Remove Element	dlist.remove(val);	Removes the first occurrence of a value.
Display List	Custom `display()` Method	Iterates through the list to print elements.
Reverse Display	Uses `reverse_iterator`	Prints elements in reverse order.
Check if Empty	dlist.empty();	Checks if the list is empty.
Clear List	dlist.clear();	Removes all elements from the list.

Comparison Table: Circular vs. Doubly Linked List in STL

Feature	Circular Linked List (std::list)	Doubly Linked List (std::list)
Supports Bidirectional Traversal	No (Manual Reset Required)	Yes (Built-in Support)
Uses STL `std::list`	Yes	Yes
Requires Custom Iterator Logic	Yes	No
Supports Reverse Iteration	No	Yes
Native STL Support	No	Yes

Circular Linked List Implementation

1. Header File (**CircularList.h**)

```
#ifndef CIRCULARLIST_H
#define CIRCULARLIST_H

#include <iostream>
#include <list>
class CircularList
{
 private:
    std::list<int> clist;
 public:
    void insert(int val);
    void remove(int val);
    void display();
    void circularDisplay(int iterations = 2);
};
   #endif    // CIRCULARLIST_H
```

---

2. Implementation File (**CircularList.cpp**)

```
#include "CircularList.h"
using namespace std;

void CircularList::insert(int val)
  {
    clist.push_back(val);
  }
void CircularList::remove(int val)
  {
    clist.remove(val);
  }
void CircularList::display()
  {
    if (clist.empty())
    {
        cout << "(empty list)" << endl;
        return;
      }
    for (auto it = clist.begin(); it != clist.end(); ++it)
    {
```

```
        cout << *it << " -> ";
      }
    cout << "(head)" << endl;
  }
void CircularList::circularDisplay(int iterations)
  {
    if (clist.empty())
     {
        cout << "(empty list)" << endl;
        return;
      }
    auto it = clist.begin();
    for (int i = 0; i < iterations * clist.size(); i++)
   {
        cout << *it << " -> ";
        ++it;
        if (it == clist.end()) it = clist.begin(); // Reset to
beginning
     }
    cout << "(repeats...)" << endl;
  }
```

---

3. Test File (**test_circular.cpp**)

```
#include "CircularList.h"
using namespace std;
int main()
{
    CircularList cl;

    cl.insert(10);
    cl.insert(20);
    cl.insert(30);
    cl.insert(40);

    cout << "Circular Linked List: ";
    cl.display();

    cout << "Simulated Circular Display: ";
    cl.circularDisplay();

    cl.remove(20);
    cout << "After Deletion: ";
    cl.display();
```

```
    system("pause");
    return 0;
}
```

Output:

```
Circular Linked List: 10 -> 20 -> 30 -> 40 -> (head)
Simulated Circular Display: 10 -> 20 -> 30 -> 40 -> 10 -> 20 -> 30 -> 40 -> (repeats...)
After Deletion: 10 -> 30 -> 40 -> (head)
```

Doubly Linked List Implementation

1. Header File (DoublyList.h)

```
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <iostream>
#include <list>
#include <iterator>

class DoublyList
{
 private:
    std::list<int> L; // The underlying STL container

 public:

    void insertFirst(int value);
    void insertEnd(int value);
    void deleteLast();

    bool insertAfter(int key, int value);
    bool insertAt(int pos, int value);
    bool deleteNode(int key);
    bool deleteAt(int pos);

    void printList() const;
    int getSize() const;
};

#endif // DOUBLYLIST_H
```

2. Implementation File (DoublyList.cpp)

```
#include "DoublyList.h"
using namespace std;

void DoublyList::insertFirst(int value)
{
    L.push_front(value);// insert element at beginning
}
```

```
void DoublyList::insertEnd(int value)
{
    L.push_back(value);// insert element at end
}

void DoublyList::deleteLast()
{
    if (!L.empty())
    {  L.pop_back();// erase element at end
    }
}

// Finds a key and inserts a value immediately after it
bool DoublyList::insertAfter(int key, int value)
{
    for (auto it = L.begin(); it != L.end(); ++it)
      {
        if (*it == key)
           {
                    ++it;
                     L.insert(it, value);
                     return true;
           }
      }
    return false;
}

// Inserts at a specific index (0-based)
bool DoublyList::insertAt(int pos, int value)
{
    if (pos < 0 || pos > (int)L.size())
      return false;

    auto it = L.begin();
    advance(it, pos);
    L.insert(it, value);
    return true;
}

// Deletes the first occurrence of a specific value
bool DoublyList::deleteNode(int key)
{
    for (auto it = L.begin(); it != L.end(); ++it)
      {
```

```
        if (*it == key)
          {
            L.erase(it);
            return true;
          }
      }
    return false;
}

// Deletes an element at a specific index
bool DoublyList::deleteAt(int pos)
{
    if (pos < 0 || pos >= (int)L.size())
      return false;

    auto it = L.begin();
    advance(it, pos);
    L.erase(it);
    return true;
}

void DoublyList::printList() const
{
    if (L.empty())
      {
        cout << "(empty list)" << endl;
        return;
      }

    for (auto it = L.begin(); it != L.end(); ++it)
          {
                cout << *it;
                auto nextIt = it;
                 ++nextIt;
              if (nextIt != L.end())
               cout << " <-> ";
           }
     cout << " <-> nullptr\n";

}

int DoublyList::getSize() const
   { return L.size(); }
```

3. Test File (test_doubly.cpp)

```
#include "DoublyList.h"
using namespace std;
int main()
{    DoublyList dl;

     //Testing insertion at front
     dl.insertFirst(30);
     dl.insertFirst(20);
     dl.insertFirst(10);
     cout << "After insertFirst (10, 20, 30): ";
     dl.printList();

     // Testing insertion at end
     dl.insertEnd(40);
     cout << "After insertEnd(40):          ";
     dl.printList();

     // Testing insertAfter
     dl.insertAfter(20, 25);
     cout << "After insertAfter(20, 25):   ";
     dl.printList();

     // Testing insertAt
     dl.insertAt(2, 22);
     cout << "After insertAt(pos 2, 22):   ";
     dl.printList();

     dl.deleteLast();// Testing deletions
     cout << "After deleteLast:               ";
     dl.printList();

     dl.deleteNode(22);
     cout << "After deleteNode(22):        ";
     dl.printList();

     dl.deleteAt(1);
     cout << "After deleteAt(pos 1):       ";
     dl.printList();

     cout << "Final List Size: " << dl.getSize() << std::endl;
      system("pause");
     return 0;
}
```

Output:

```
After insertFirst (10, 20, 30): 10 <-> 20 <-> 30 <-> nullptr
After insertEnd(40):          10 <-> 20 <-> 30 <-> 40 <-> nullptr
After insertAfter(20, 25):   10 <-> 20 <-> 25 <-> 30 <-> 40 <-> nullptr
After insertAt(pos 2, 22):   10 <-> 20 <-> 22 <-> 25 <-> 30 <-> 40 <-> nullptr
After deleteLast:               10 <-> 20 <-> 22 <-> 25 <-> 30 <-> nullptr
After deleteNode(22):        10 <-> 20 <-> 25 <-> 30 <-> nullptr
After deleteAt(pos 1):       10 <-> 25 <-> 30 <-> nullptr
Final List Size: 3
```

Computer Engineering College
Second Class

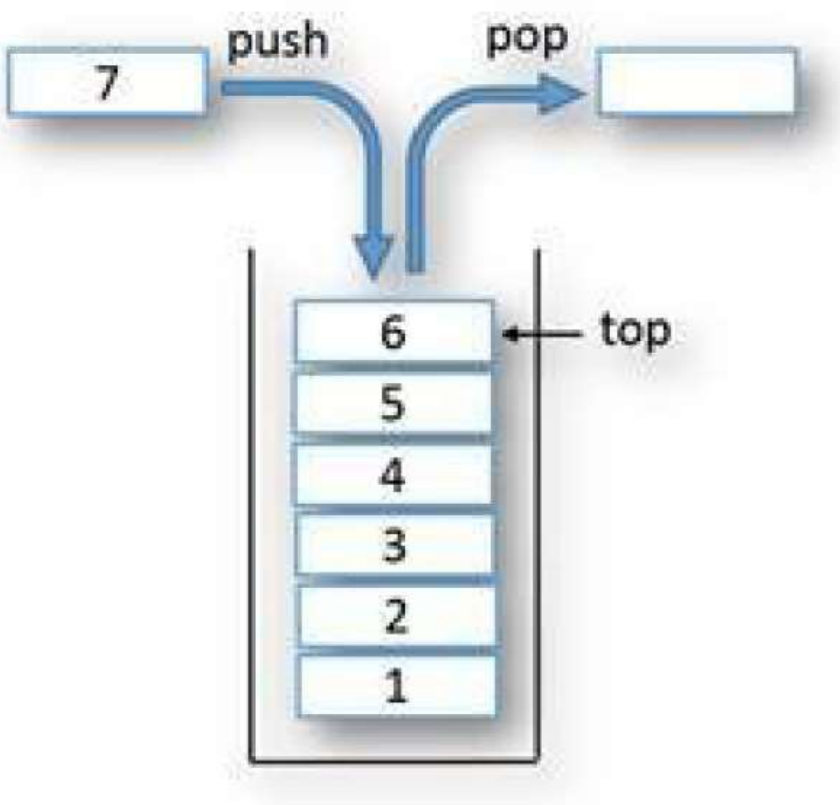
Data Structure and Algorithms

Lecture Three : Stacks

Dr. Amthal Kh. Mousa **Lec. Zainab M. Fadhil**

What is a Stack?

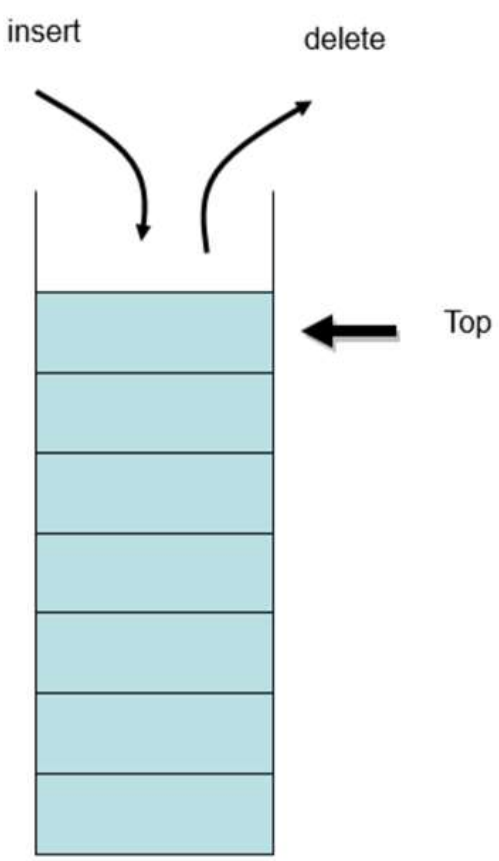
- A **Stack** is an abstract data type that serves as a collection of elements, with two main principal operations:
- **Push:** Adds an element to the collection.
- **Pop:** Removes the most recently added element that was not yet removed.
- It follows the **LIFO (Last-In, First-Out)** principle. Imagine a stack of books: the last book you place on the pile is the first one you must take off.

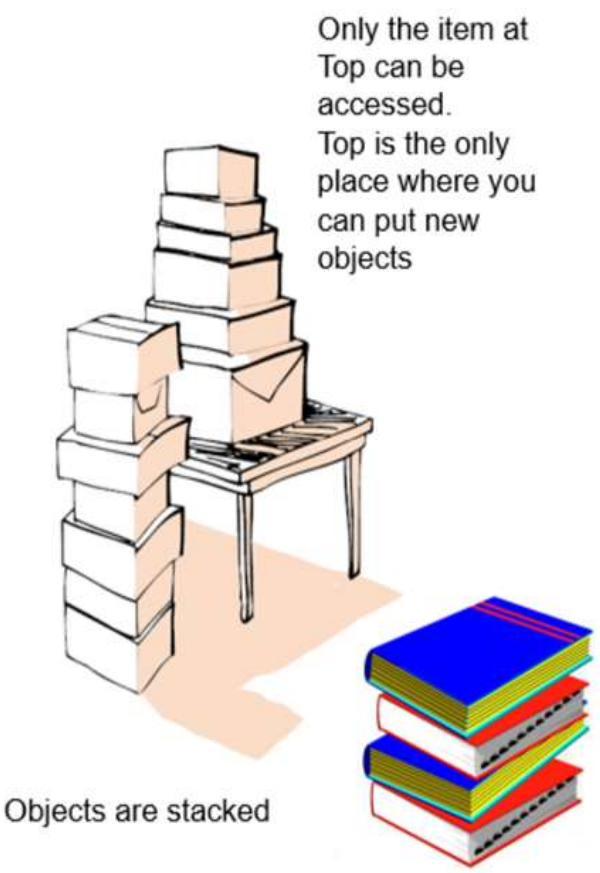


Core Operations in C++ STL

- To use a stack in C++, we use the <stack> header.

Method	Description
push(val)	Inserts val at the top.
pop()	Removes the top element.
top()	Returns a reference to the top element.
empty()	Returns true if the stack is empty.
size()	Returns the number of elements.

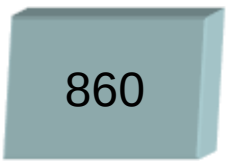




Insert Operation

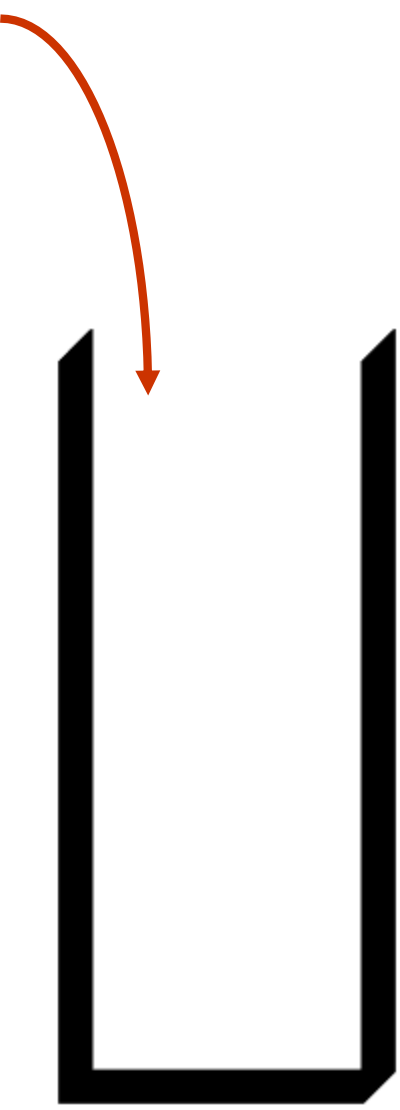
Insert 8

Insert 860



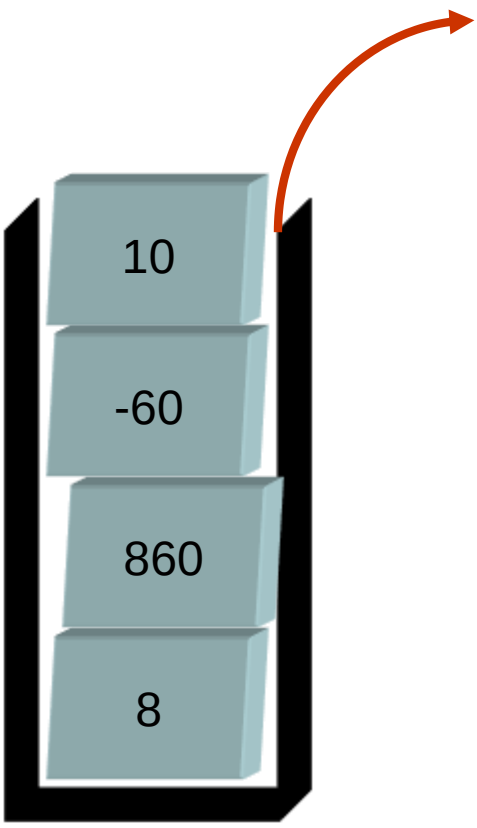
Insert -60

Insert 10



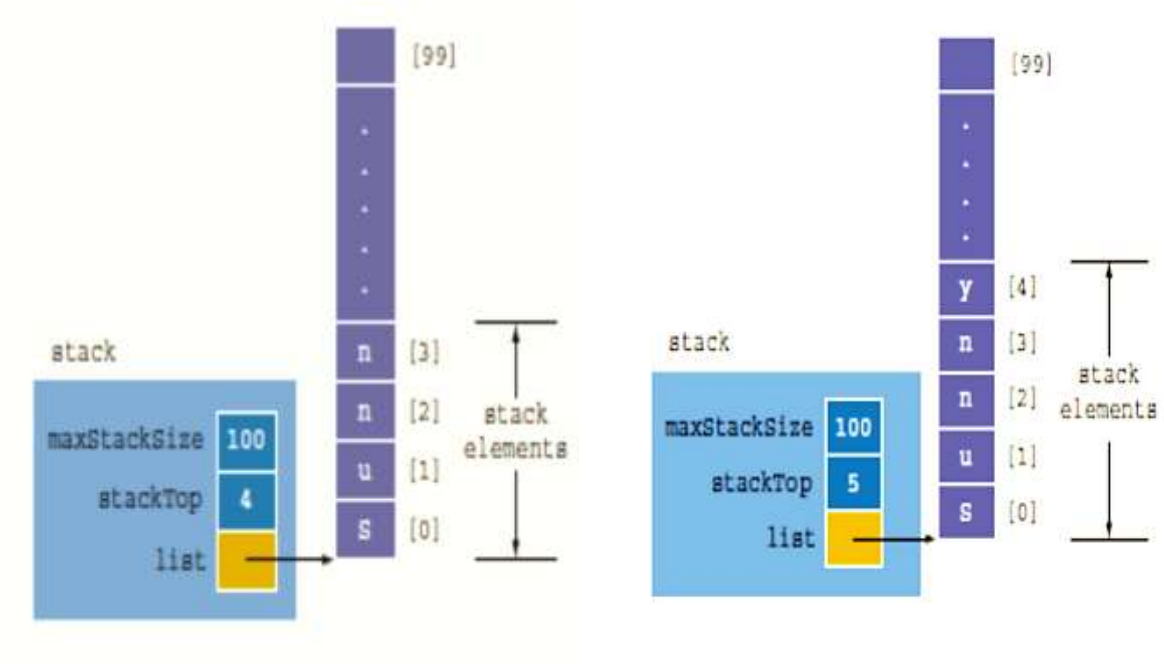
A stack is a dynamic structure. It changes as elements are added to and removed from it.

Delete Operation



Stack implementation

Array stacks:



- array size is fixed
- in the array (linear) representation of a stack, only a fixed number of elements can be pushed onto the stack.
- If number of elements to be pushed exceeds the size of the array, the program may terminate in an error.

Linked Implementation of Stacks

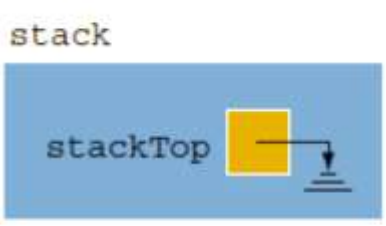


Fig :Empty Linked stack

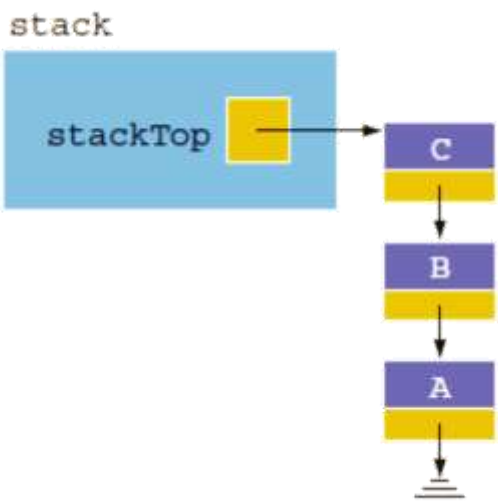


Fig: nonempty linked stack

Implementation: Basic Stack Operations

- implementation a simple stack to manage a history of strings (like a browser's "back" button).

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;
int main()
{
    stack<string> history;

    // Adding pages to the stack
cout<<"pushing (google.com)\n";
    history.push("google.com");

cout<<"pushing (github.com)\n";
    history.push("github.com");

cout<<"pushing (stackoverflow.com)\n";
    history.push("stackoverflow.com");

    cout << "\nCurrently at: " << history.top() << endl;
// stackoverflow.com

    // User clicks 'Back'
cout<<" pop operation...\n";
    history.pop();

    if (!history.empty())
    {
        cout << "Back to: " << history.top() << endl;
    }

cout<<"\n pop operation...\n";
    history.pop();

    if (!history.empty())
      {
        cout << "Back to: " << history.top() << endl;
      }

system("pause");
    return 0;
}
```

Output:

```
pushing (google.com)
pushing (github.com)
pushing (stackoverflow.com)

Currently at: stackoverflow.com
 pop operation...
Back to: github.com

 pop operation...
Back to: google.com
```

Reverse-Polish Notation

Normally, mathematics is written using what we call *in-fix* notation:

$$(3 + 4) \times 5 - 6$$

The operator is placed between to operands

One weakness: parentheses are required

$$\begin{aligned}(3 + 4) \times 5 - 6 &= 29\\ 3 + 4 \times 5 - 6 &= 17\\ 3 + 4 \times (5 - 6) &= -1\\ (3 + 4) \times (5 - 6) &= -7\end{aligned}$$

Alternatively, we can place the operands first, followed by the operator:

$$\begin{aligned}(3 + 4) \times 5 - 6\\ 3\ 4\ +\ 5\ \times\ 6\ -\end{aligned}$$

Parsing reads left-to-right and performs any operation on the last two operands:

$$\begin{aligned}&3\ 4\ +\ 5\ \times\ 6\ -\\ &7\quad 5\ \times\ 6\ -\\ &35\quad 6\ -\\ &29\end{aligned}$$

Other examples:

$$\begin{aligned}&3\ 4\ 5\ \times\ +\ 6\ -\\ &3\quad 20\quad +\ 6\ -\\ &23\quad\quad 6\ -\\ &17\end{aligned}$$

$$\begin{aligned}&3\ 4\ 5\ 6\ -\ \times\ +\\ &3\ 4\quad -1\quad \times\ +\\ &3\quad\quad -4\quad\quad +\\ &-1\end{aligned}$$

Advanced Application: Reverse Polish Notation (RPN)

- **Reverse Polish Notation** (also called Postfix Notation) is a mathematical notation in which every operator follows all of its operands.
- It is used in calculators and compilers because it is entirely unambiguous and does not require parentheses.

•**Infix (Standard):** (3 + 4) * 5

•**RPN (Postfix):** 3 4 + 5 *

Benefits:

- No ambiguity and no brackets are required
- It is the same process used by a computer to perform computations:
  - operands must be loaded into registers before operations can be performed on them
- Reverse-Polish can be processed using stacks

The Evaluation Algorithm

The stack is the perfect tool for RPN because it allows us to store operands until we encounter an operator.

Postfix Evaluation Algorithm

Inputs: postfix expression string
Output: number representing answer
Private data: a stack

1. Start with the left-most token.
2. If the token is a number:
   a. Push it onto the stack
3. Else if the token is an operator:
   a. Pop the top value into a variable called v2, and the second-to-top value into v1.
   b. Apply operator to v1 and v2 (e.g., v1 / v2)
   c. Push the result of the operation on the stack
4. If there are more tokens, advance to the next token and go back to step #2

5. After all tokens have been processed, the top # on the stack is the answer!

7 6 * 5 +

Evaluate the following Postfix (reverse-Polish) expression using a stack:

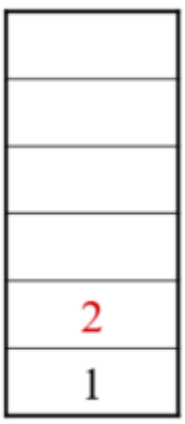
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Push 1 onto the stack

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Push 1 onto the stack

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Push 3 onto the stack

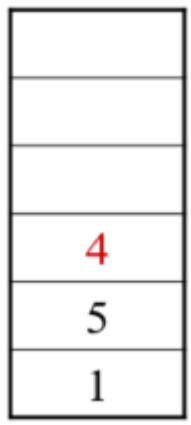
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Pop 3 and 2 and push 2 + 3 = 5

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Push 4 onto the stack

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Push 5 onto the stack

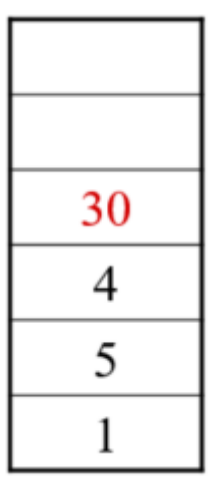
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Push 6 onto the stack

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Pop 6 and 5 and push 5 × 6 = 30

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Pop 30 and 4 and push 4 – 30 = –26

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Push 7 onto the stack

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Pop 7 and –26 and push –26 × 7 = –182

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Pop –182 and 5 and push –182 + 5 = –177

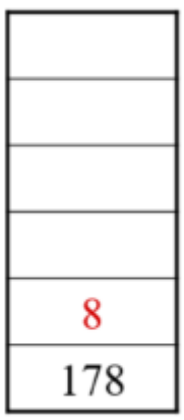
1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Pop –177 and 1 and push 1 – (–177) = 178

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Push 8 onto the stack

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Push 1 onto the stack

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Pop 9 and 8 and push 8 × 9 = 72

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Pop 72 and 178 and push 178 + 72 = 250

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +

Thus

$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$

evaluates to the value on the top: 250

The equivalent in-fix notation is

$$((1 - ((2 + 3) + ((4 - (5 \times 6)) \times 7))) + (8 \times 9))$$

We reduce the parentheses using order-of-operations:

$$1 - (2 + 3 + (4 - 5 \times 6) \times 7) + 8 \times 9$$

Incidentally,

$$1 - 2 + 3 + 4 - 5 \times 6 \times 7 + 8 \times 9 = -132$$

which has the reverse-Polish notation of

$$1\ 2\ -\ 3\ +\ 4\ +\ 5\ 6\ 7\ \times\ \times\ -\ 8\ 9\ \times\ +$$

For comparison, the calculated expression was

$$1\ 2\ 3\ +\ 4\ 5\ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ 9\ \times\ +$$

Postfix Evaluation Code:

```
#include <iostream>
#include <stack>
#include <string>
#include <cstdlib> // For atof

using namespace std;

double evalRPN(const char* tokens[], int size)
{
    stack<double> s;

    for (int i = 0; i < size; ++i)
    {
        string t = tokens[i];

        // An operator is a single character and matches one of the symbols.
        // This prevents "-5" from being seen as the "-" operator.

        if (t.length() == 1 && (t == "+" || t == "-" || t == "*" || t == "/"))
        {
            if (s.size() < 2)
              {
                cout << "Error: Invalid RPN expression" << endl;
                return 0;
               }
            double val2 = s.top(); s.pop();
            double val1 = s.top(); s.pop();

            if (t == "+") s.push(val1 + val2);
            else if (t == "-") s.push(val1 - val2);
            else if (t == "*") s.push(val1 * val2);
            else if (t == "/")
              {
                if (val2 == 0)
                   {
                     cout << "Error: Division by zero" << endl;
                     return 0;
                   }
                s.push(val1 / val2);
              }
          }
        else
        {   // Convert token to double
            s.push(atof(tokens[i]));
        }
    }
    return s.top();
}

int main()
{
    // Represents: (-10.5 + 2.5) * 0.5  => -8 * 0.5 = -4
    const char* rpn[] = {"-10.5", "2.5", "+", "0.5", "*"};
            // const char* rpn[] = {"-10.5", "+"};
    int size = sizeof(rpn) / sizeof(rpn[0]);

    cout << "RPN Result: " << evalRPN(rpn, size) << endl;

    system("pause");
    return 0;
}
```

Output1:

```
RPN Result: -4.00
```

Output2:

If we enter invalid expression:

```
// const char* rpn[] = {"-10.5", "2.5", "+", "0.5", "*"};
    const char* rpn[] = {"-10.5", "+"};
```

The output will be:

```
Error: Invalid RPN expression
RPN Result: 0.00
```

Decimal to Binary Conversion using Stacks

Objectives:

- To implement a **LIFO (Last-In, First-Out)** data structure using the C++ and STL (std::stack).
- To convert a decimal number to its equivalent binary form.

Implementation:

- Input: decimal number.
- Output: binary number representing answer.
- Because the binary bits are generated from the **Least Significant Bit (LSB)** to the **Most Significant Bit (MSB)**, we need a technique to flip them around to read the number correctly. The stack's **LIFO** nature handles this perfectly.

Algorithm:

1. **Start.**
2. Initialize an empty stack S of type Integer.
3. Input a decimal number N.
4. If N is 0, display "0" and go to Step 7.
5. **Phase 1 (Conversion):** While N>0:
    - Calculate the remainder R = N mod 2.
    - **Push** R onto the stack S.
    - Update N by performing integer division: N=N/2.
6. **Phase 2 (Output):** While the stack S is **not empty**:
    - Access the **Top** element of the stack.
    - Display the element.
    - **Pop** the element from the stack.
7. **End.**

Code:

```
#include <iostream>
#include <stack>
using namespace std;
// Function to convert decimal to binary using a stack
void decimalToBinary(int n)
{
    // A stack to store the remainders (0s and 1s)
    stack<int> s;

    // Handle the case for 0 explicitly
    if (n == 0)
     {
        cout << "Binary: 0" << endl;
        return;
     }

 // Step 1: Divide and push remainders
    int originalNumber = n;
    while (n > 0)
     {
        int remainder = n % 2;
        s.push(remainder);
        n = n / 2;
     }

  // Step 2: Pop and display the binary digits
    cout << "Decimal: " << originalNumber << " -> Binary: ";
    while (!s.empty())
     {
        cout << s.top(); // Print the top element
        s.pop();         // Remove the top element
     }
    cout << endl;
}
```

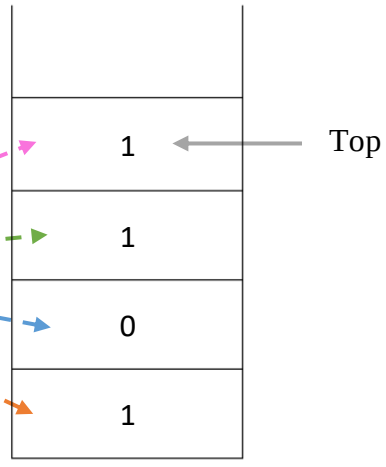
```
int main()
{
    int num;
    cout << "Enter a decimal number: ";
    cin >> num;

    decimalToBinary(num);

    system("pause");
    return 0;

}
```

Sample Output:

```
Enter a decimal number: 13
Decimal: 13 -> Binary: 1101
```

Trace:

Step	Division	Remainder	Stack (Bottom to Top)
1	13 / 2	**1**	[1]
2	6 / 2	**0**	[1, 0]
3	3/ 2	**1**	[1, 0, 1]
4	1/ 2	**1**	[1, 0, 1, 1]



Stack

Balanced Parentheses using Stack

Requirements:

- Use std::stack<char>
- Support symbols: () {} []
- Ignore non-bracket characters
- Return true only if properly balanced

Example:

Input: {(a+b)*[c-d]}

Output: Balanced

Algorithm ParenMatch(X, n):
Input: An array X of n tokens, each of which is either a grouping symbol, a variable, an arithmetic operator, or a number
Output: **true** if and only if all the grouping symbols in X match

```
Let S be an empty stack
for i ← 0 to n − 1 do
    if X[i] is an opening grouping symbol then
        S.push(X[i])
    else if X[i] is a closing grouping symbol then
        if S.empty() then
            return false          {nothing to match with}
        if S.top() does not match the type of X[i] then
            return false          {wrong type}
        S.pop()
if S.empty() then
    return true                   {every symbol matched}
else
    return false                  {some symbols were never matched}
```

Code:

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isBalanced(const string& expression)
{
    std::stack<char> st;
    for (char c : expression)
    {
        if (c == '(' || c == '{' || c == '[')
            st.push(c);
        else if (c == ')' || c == '}' || c == ']')
        {
            if (st.empty()) return false;

            char top = st.top(); st.pop();

            if ((top == '(' && c != ')') ||
                (top == '{' && c != '}') ||
                (top == '[' && c != ']'))
                return false;
        }
    }
    return st.empty();
}

int main()

{    string expr;

     cout << "Enter expression: ";
         //getline(cin, expr);
      cin>> expr;
     cout << (isBalanced(expr) ? "Balanced\n" : "Not Balanced\n");

    return 0;
}
```

Computer Engineering College
Second Class

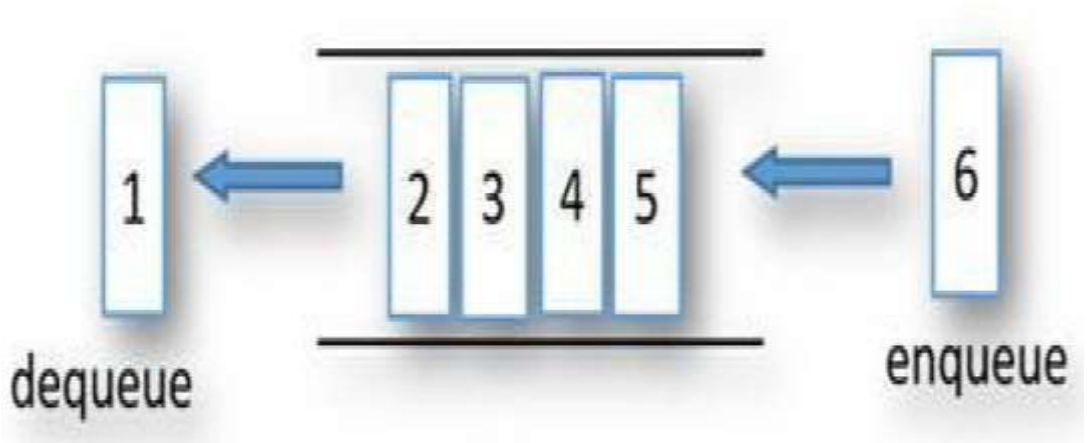
Data Structure and Algorithms

Lecture Four: Queue

Dr. Amthal Kh. Mousa **Lec. Zainab M. Fadhil**

What is a Queue?

- A queue is a data structure in which the elements are added at one end, called the rear, and deleted from the other end, called the front; a **F**irst **I**n **F**irst **O**ut (**FIFO**) data structure.



- Think of it like a line of people waiting for service: the first person to join the line is the first one to be served.
- Rear (Back): The end where new elements are added.
- Front: The end where elements are removed.
- Logic: The element that has been in the queue the longest is at the front.

Real-World & Technical Applications

- **System Programming:** Operating systems use queues to manage processes ready for execution.
- **Networking & Buffers:** Act as a "holding area" (buffer) for messages between two processes, programs, or systems.
- **Client-Server Models:** Servers (web, database, mail) use queues to handle multiple incoming client requests in order of arrival.
- **File Transfers:** SFTP clients queue files waiting for transfer.
- **Everyday Life:** Lines at banks, grocery stores, and airport security.

Applications

- For example, in downloading these presentations from the web server, those requests not currently being downloaded are marked as “Queued”

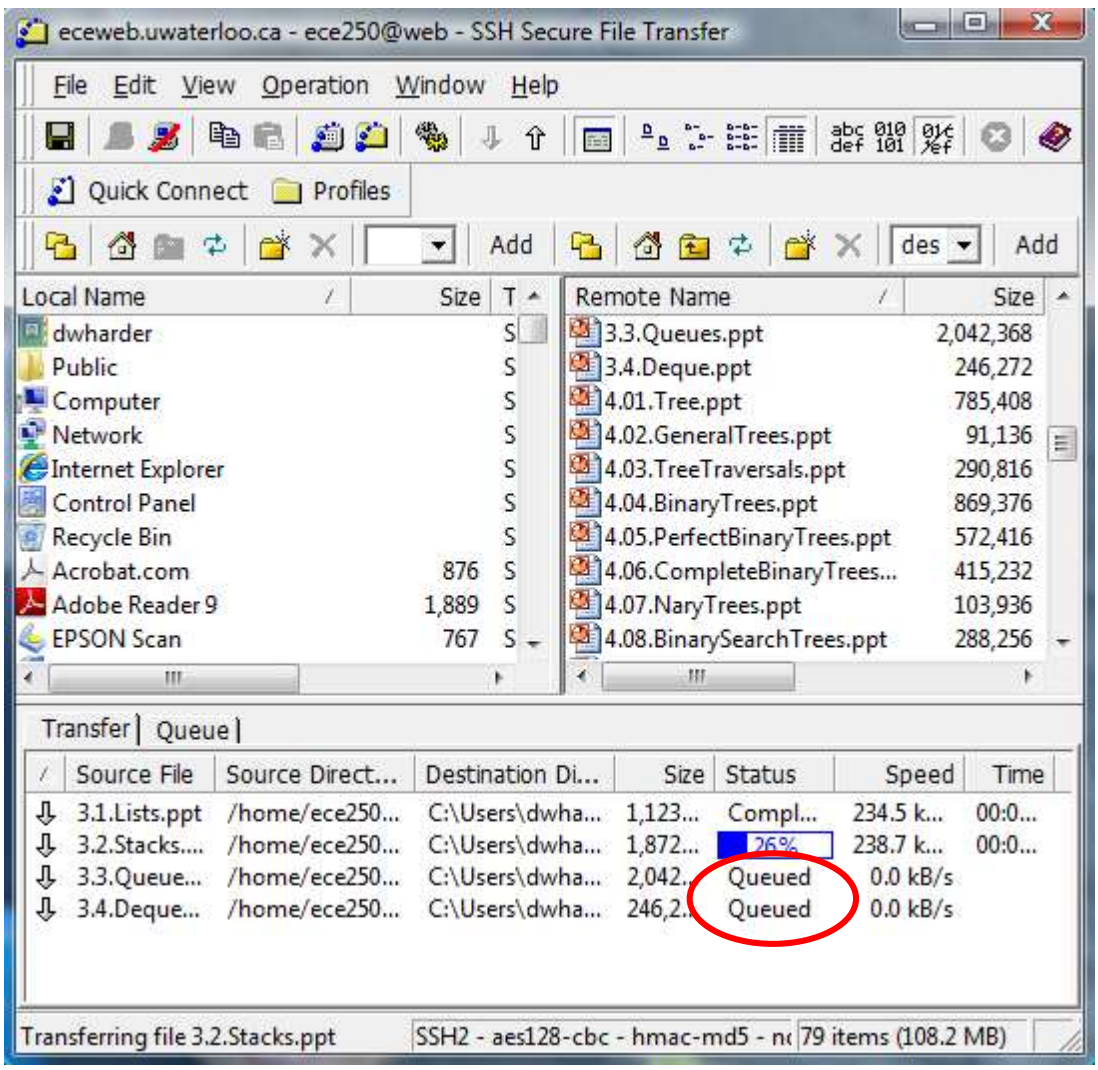


Figure 2. The SFTP client showing the queuing of files waiting for transfer

The STL Queue Operations:

- size(): Return the number of elements in the queue.
- empty(): Return true if the queue is empty and false otherwise.
- push(*e*): Enqueue *e* at the rear of the queue.
- pop(): Dequeue the element at the front of the queue.
- front(): Return a reference to the element at the queue's front.
- back(): Return a reference to the element at the queue's rear.

- The insertion and erase operations are significantly restricted:
  1. Objects are always pushed onto the back/rear of the queue,
  2. The front of the queue is the object that was least recently pushed onto the queue, and
  3. When an object is popped from the queue, it erases the current front of the queue.

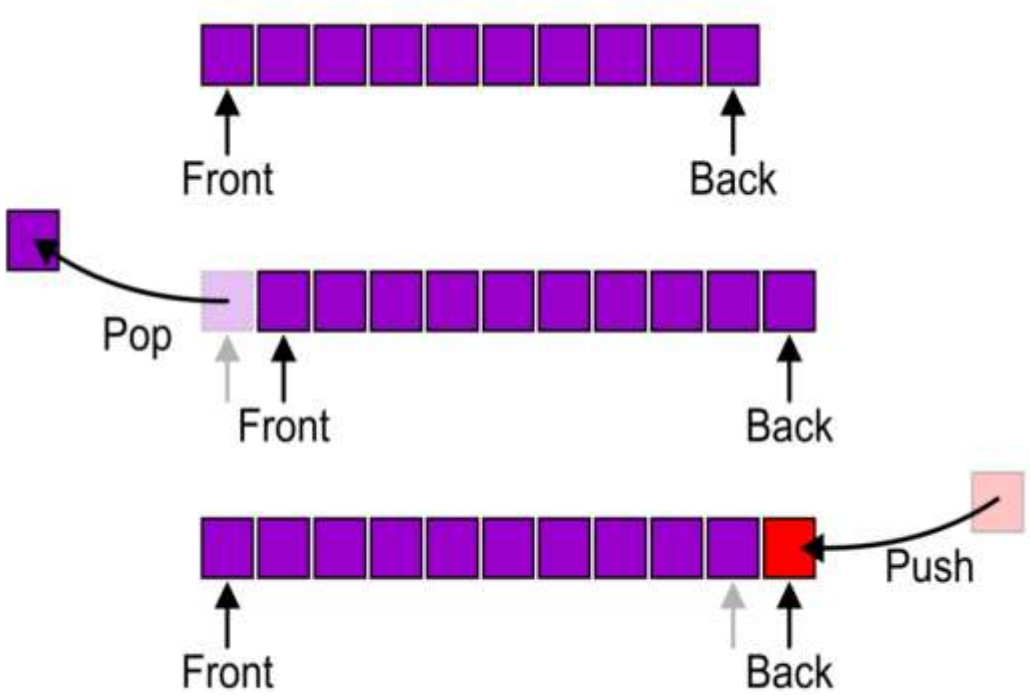
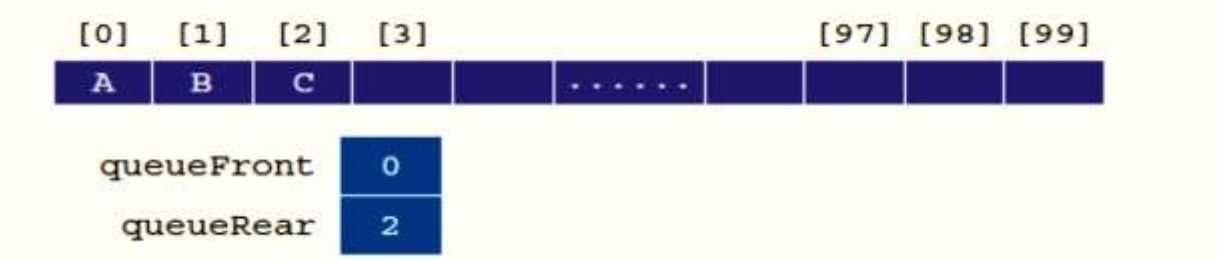


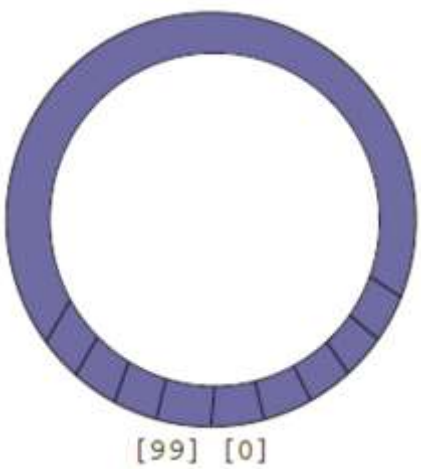
Figure 1. The front, push, and pop operations on a queue.

Implementations:

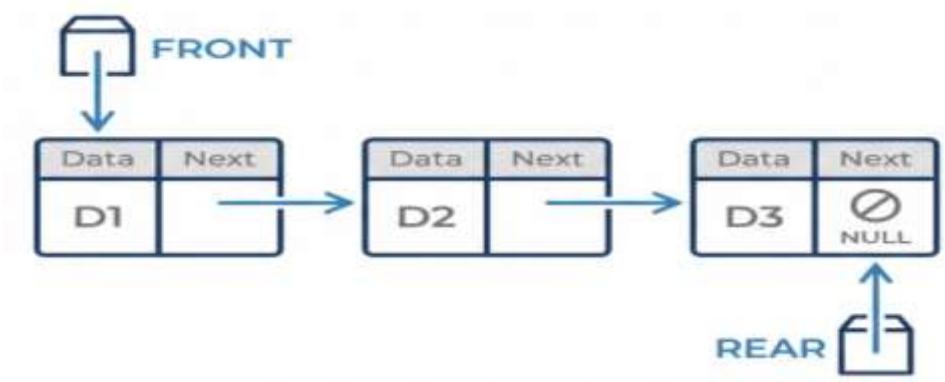
- **Array-Based Queue:** Simple to implement but can be inefficient. Removing an item from the front may require shifting all other items forward.



- **Circular queue** ((Improved Array)): A more efficient array implementation where the "Rear" wraps around to the beginning of the array when it reaches the end, eliminating the need to shift items.
  - **Formula for circular increment:** rear=(rear+1) % (size)



- **linked queue**: Uses a linked list. This allows the queue to grow dynamically without a fixed size.



Implementing a Queue with a Circularly Linked List

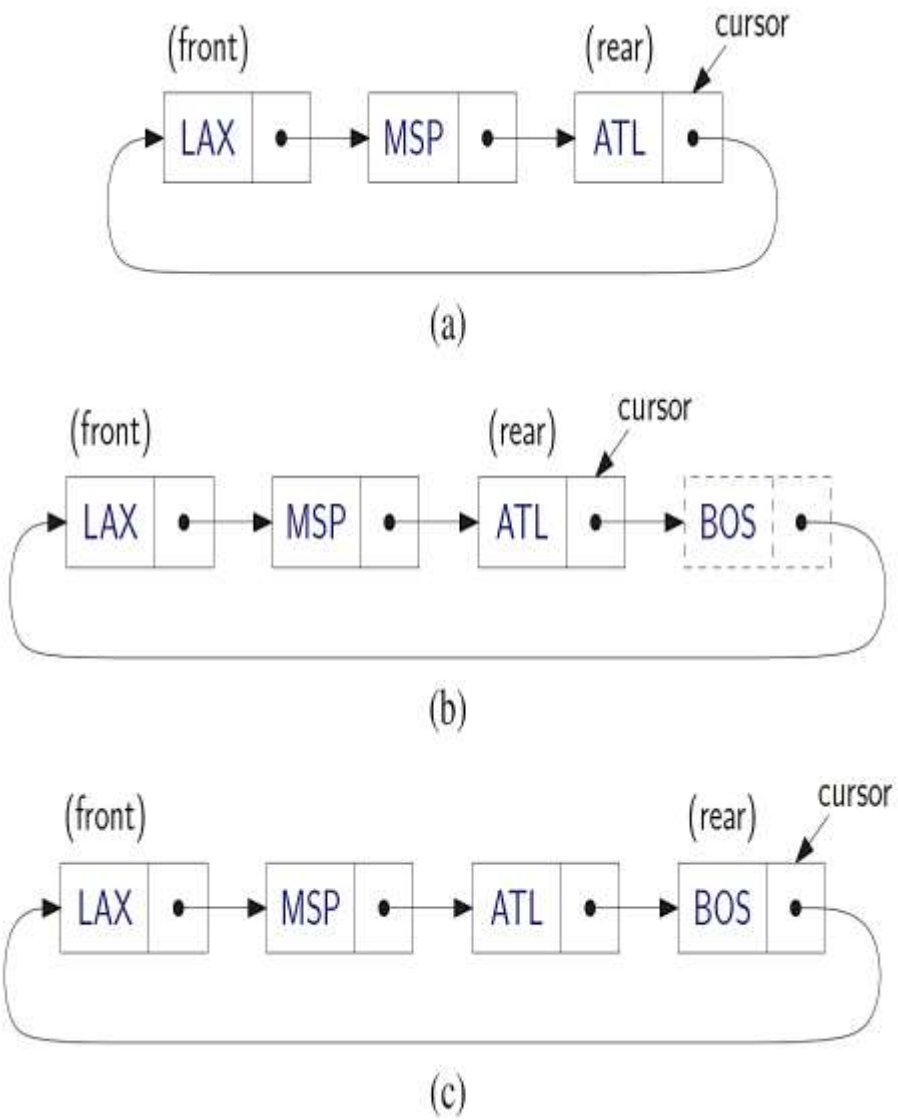


Figure : Enqueueing “BOS” into a queue represented as a circularly linked list: (a) before the operation; (b) after adding the new node; (c) after advancing the cursor.

A Queue in the STL

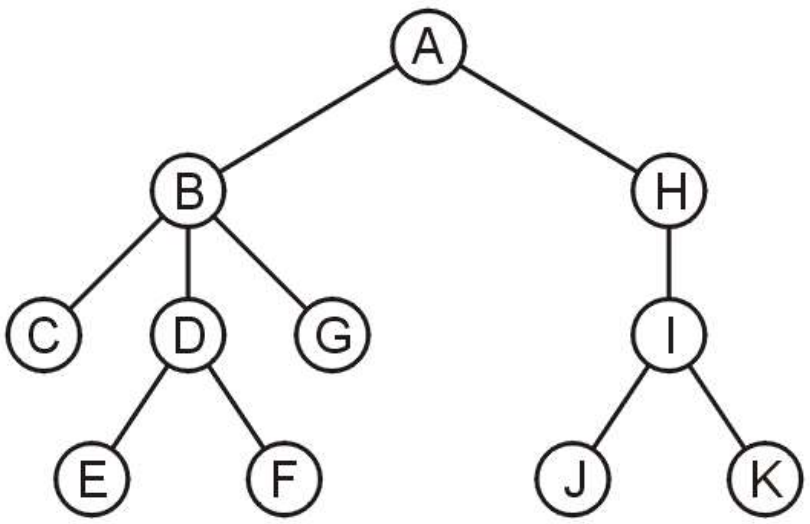
```
#include <iostream>
#include <queue>
using namespace std;

int main()
{
 std::queue<int>  iqueue;// queue of ints

 iqueue.push(10); // add item to rear
 iqueue.push(20);
 iqueue.push(30);
 iqueue.push(40);

 cout <<"queue front = "<< iqueue.front()<<endl;
    iqueue.pop();

   if (! iqueue.empty() )
              cout <<"size of queue = "<<
               iqueue.size()<<endl;

     // Processing remaining items
    while (! iqueue.empty() )
    {
        cout << iqueue.front()<<endl;
        iqueue.pop();
    }

return 0;
}
```

Output:

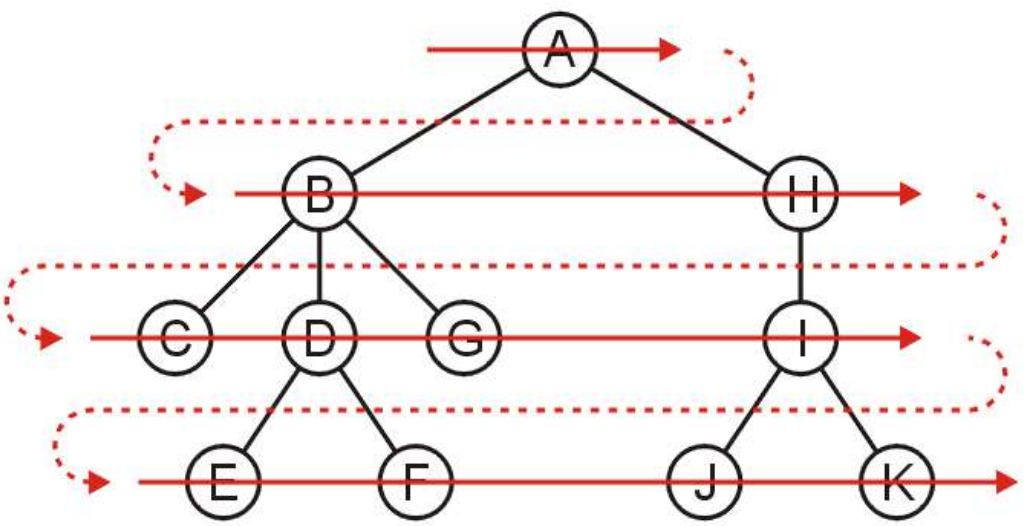
```
queue front = 10
size of queue = 3
20
30
40
```

Application

- Another application is performing a **breadth-first traversal** of a directory tree
  - Consider searching the directory structure



- We would rather search the more shallow directories first then plunge deep into searching one sub-directory and all of its contents
- One such search is called a *breadth-first traversal*
  - Search all the directories at one level before descending a level

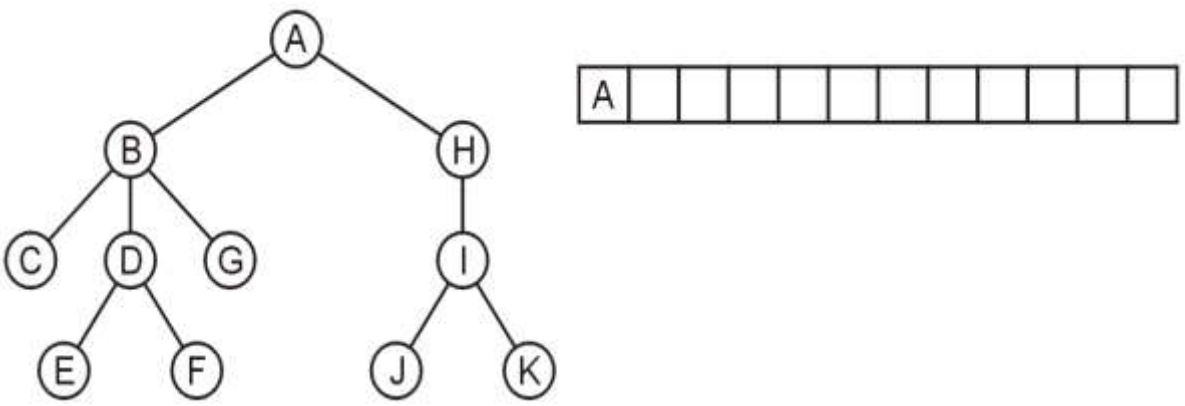


The easiest implementation is:

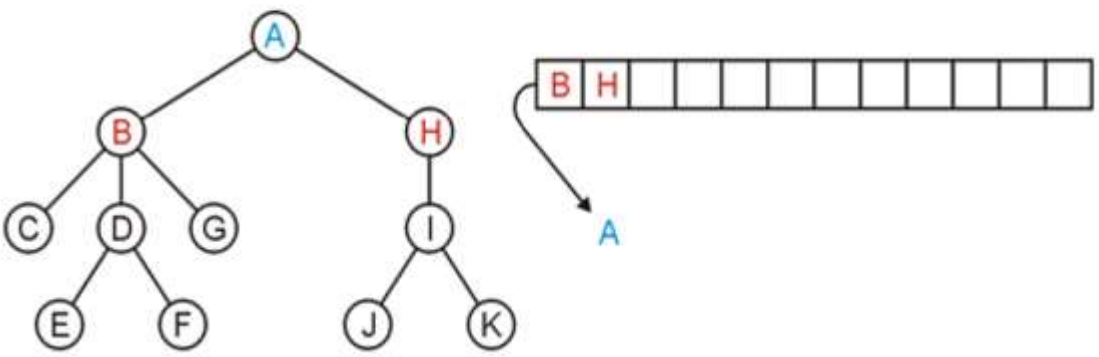
- Place the root directory into a queue
- While the queue is not empty:
  - Pop the directory at the front of the queue
  - Push all of its sub-directories into the queue

The order in which the directories come out of the queue will be in breadth-first order

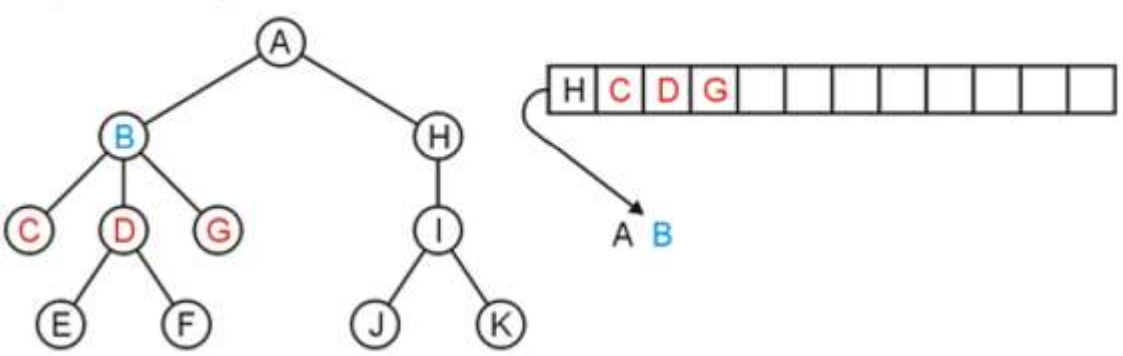
Push the root directory A



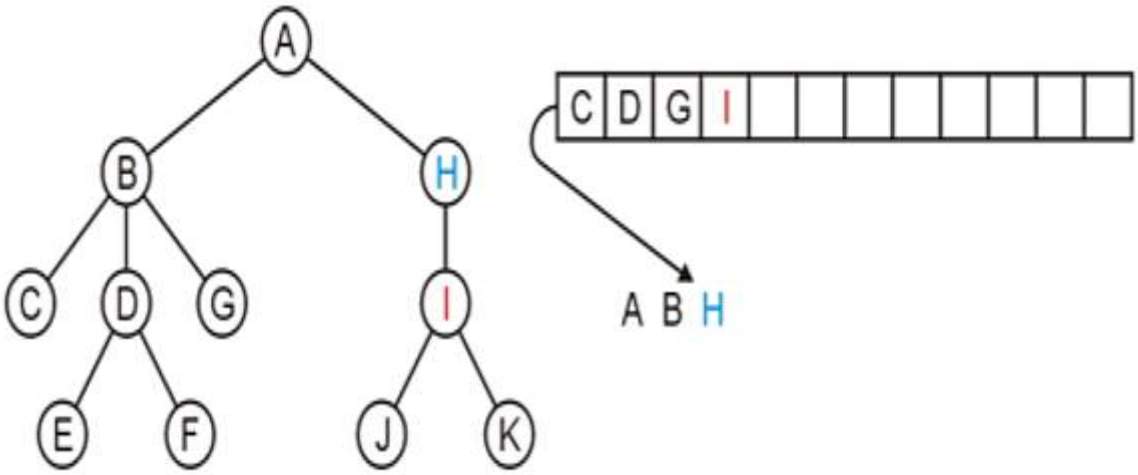
Pop A and push its two sub-directories: B and H



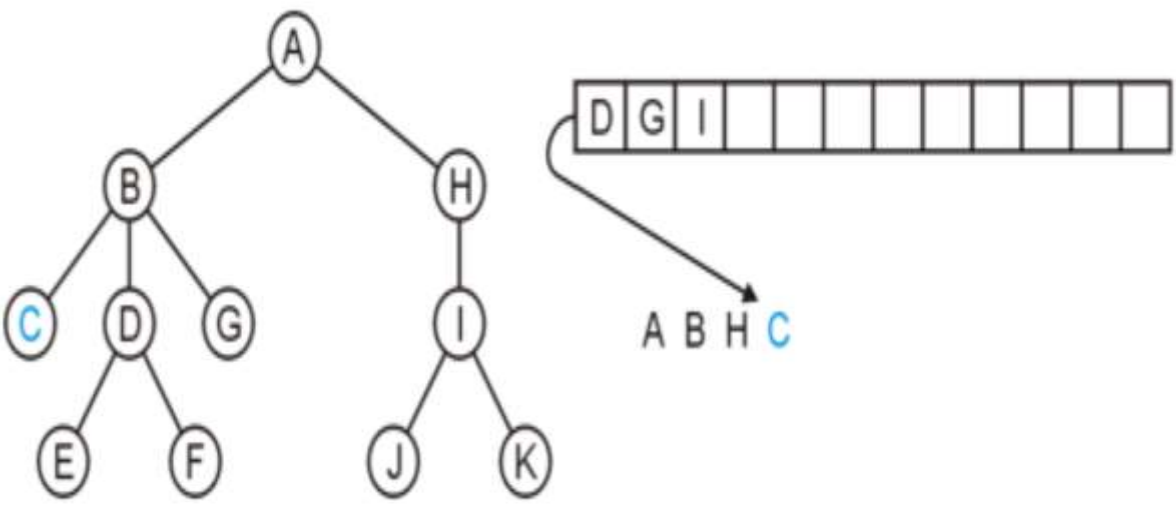
Pop B and push C, D, and G



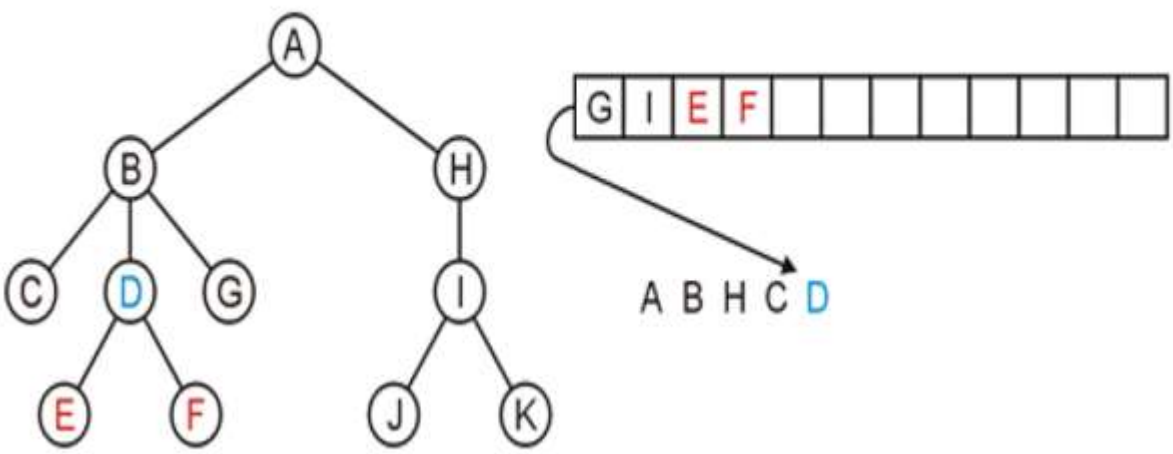
Pop H and push its one sub-directory I



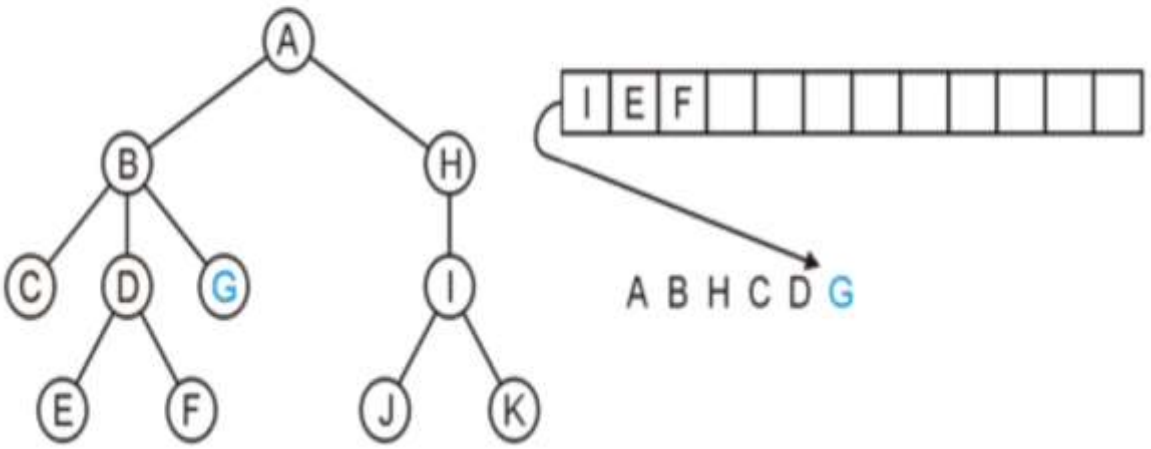
Pop C: no sub-directories



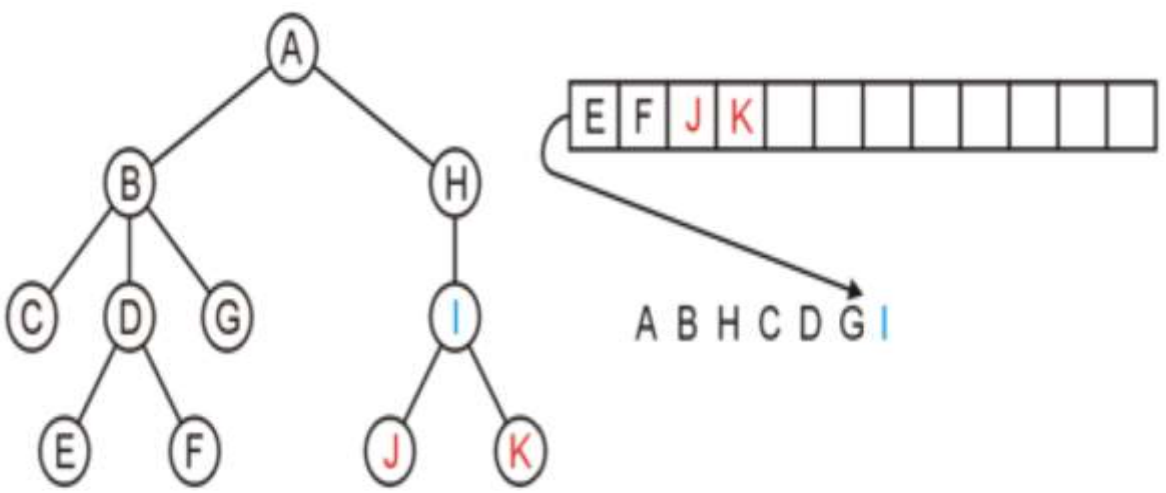
Pop D and push E and F



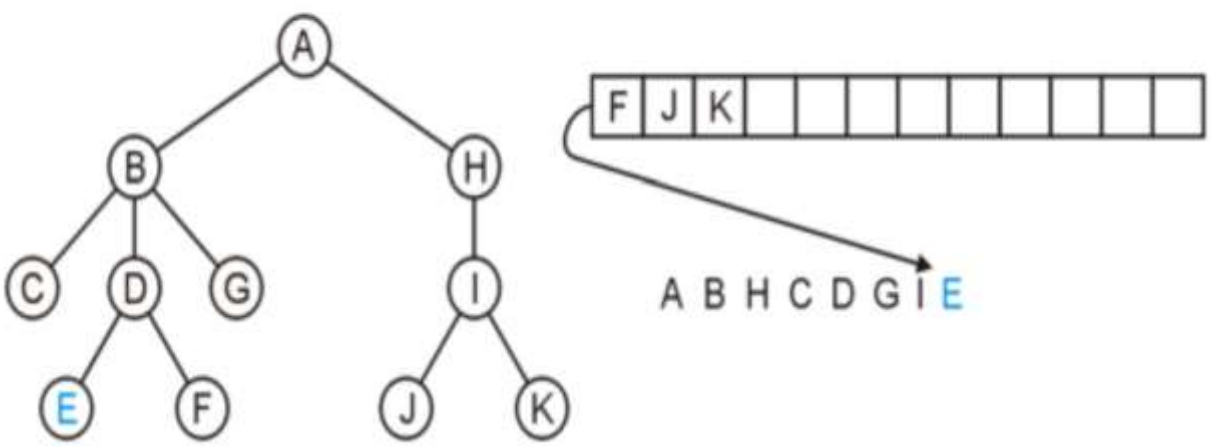
Pop G



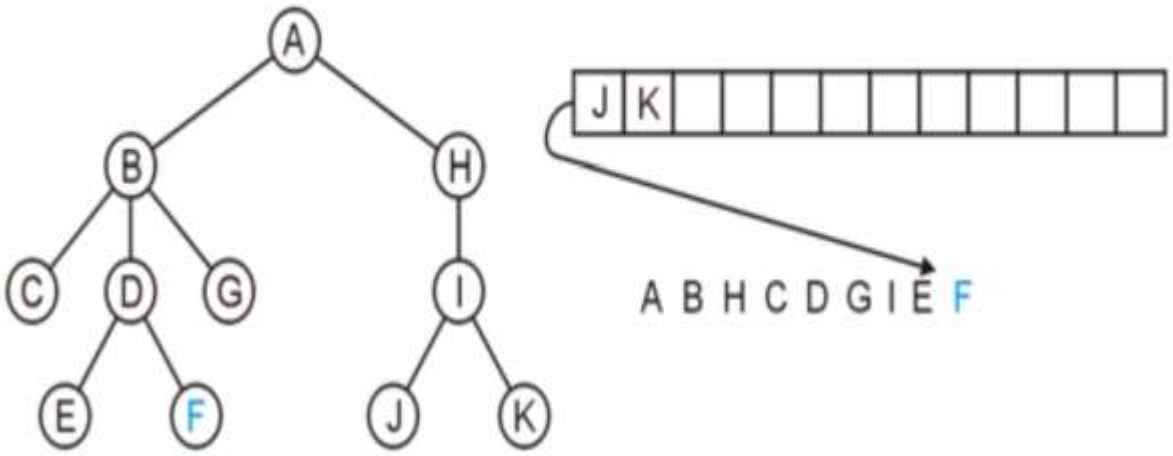
Pop I and push J and K



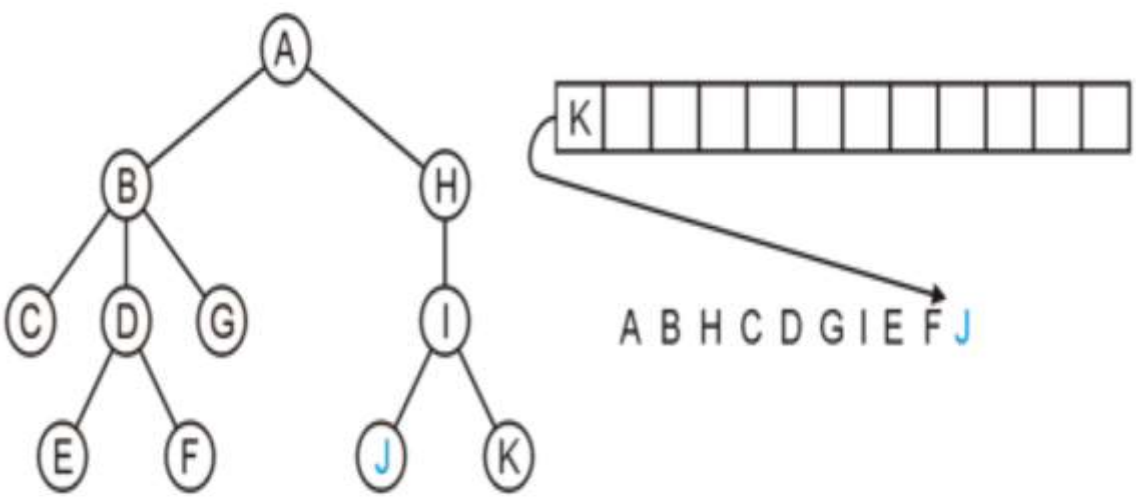
Pop E



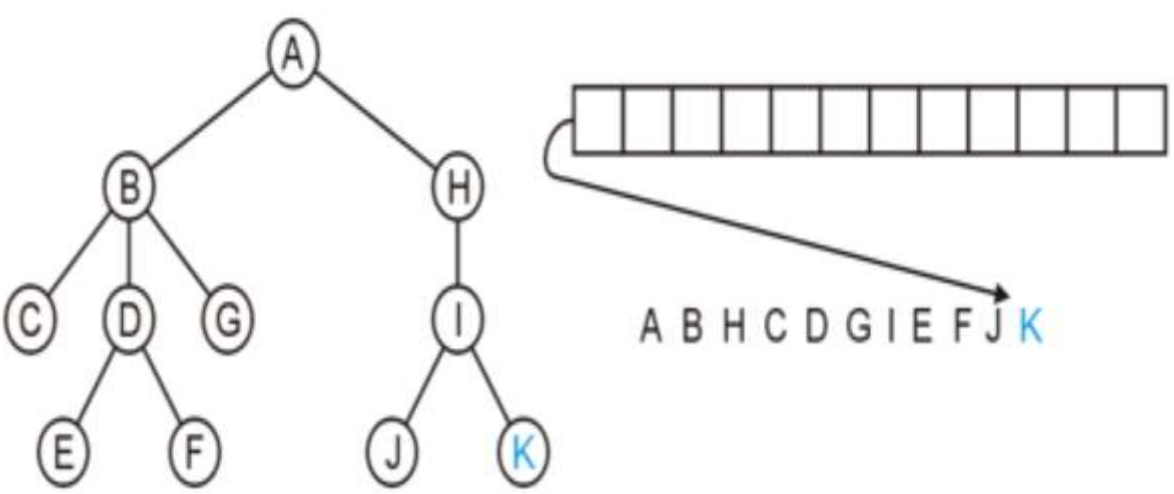
Pop F



Pop J



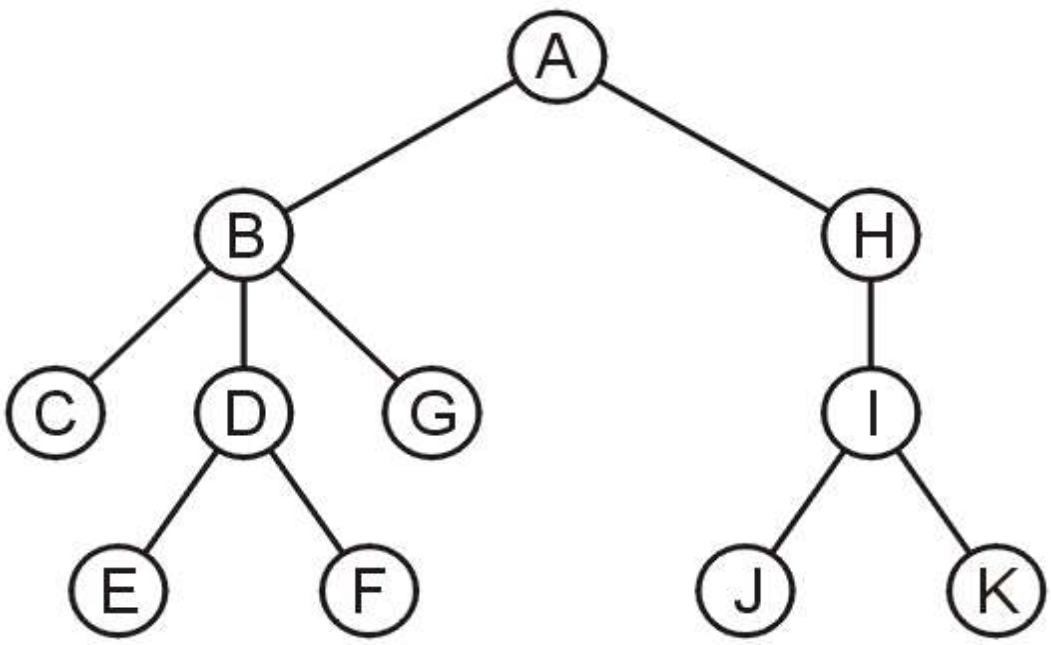
Pop K and the queue is empty



The resulting order

A B H C D G I E F J K

is in breadth-first order:



Double-Ended Queue (Deque)

- Consider now a queue-like data structure that supports insertion and deletion at both the front and the rear of the queue. Such an extension of a queue is called a **double-ended queue**, or **deque**, which is usually pronounced “deck” to avoid confusion with the dequeue function of the regular queue ADT.
- **Purpose and suitability**
  - std::deque is a container that provides rapid insertions and deletions at both its beginning and end. Its primary strengths are as follows:
  - Efficient O(1) insertions and deletions at both ends
  - Dynamic size with no need for manual memory management
  - Fairly good cache performance for front and back operations
- **Ideal use cases**
  - **Fast insertion at both ends**: it is ideal for scenarios requiring operations on both ends.
  - **General purpose queue and stack**: Due to its double-ended nature, std::deque can serve as a queue (FIFO data structure) and a stack (LIFO data structure). It’s versatile in this regard, unlike other containers specializing in one or the other.
  - **Random access:** it offers constant-time random access to elements, making it suitable for applications that need to access elements by index.
  - **Dynamic growth**: it is useful for situations where the dataset might expand unpredictably on both ends.
  - **Buffered data streams**: When building applications such as media players that buffer data, std::deque can be helpful. As data is consumed from the front (played), new data can be buffered at the back without reshuffling the entire dataset.
  - **Sliding window algorithms**: In algorithms that require a *sliding window* approach, where elements are continuously added and removed from both ends of a data structure, std::deque offers an efficient solution.
  - **Adaptable to other data structures**: With its flexibility, std::deque can easily be adapted into other custom data structures. For example, a balanced tree or a specific kind of priority queue might leverage the capabilities of std::deque.
  - **Undo/redo mechanisms with a limit**: For software applications that provide undo and redo functionality but want to limit the number of stored actions, a std::deque can efficiently handle adding new actions and automatically removing the oldest ones.

deque Operations:

- size(): Return the number of elements in the deque.
- empty(): Return true if the deque is empty and false otherwise.
- push_front(*e*): Insert *e* at the beginning the deque.
- push_back(*e*): Insert *e* at the end of the deque.
- pop_front(): Remove the first element of the deque.
- pop_back(): Remove the last element of the deque.
- front(): Return a reference to the deque's first element.
- back(): Return a reference to the deque's last element.

STL Implementation

```
#include <iostream>
#include <deque>
using namespace std;

int main()
{
 std::deque<int>  dq;// queue of ints

 dq.push_front(20);
 dq.push_back(30);
 dq.push_back(40);
 dq.push_back(50);
 dq.push_front(10);

 cout <<"queue front = "<< dq.front()<<endl;
   cout <<"queue back = "<< dq.back()<<endl;

   if (! dq.empty() )
     cout <<"size of queue = "<< dq.size()<<endl;

   cout<<"\nelements in the queue :\n";
   for(auto it = dq.begin(); it != dq.end(); ++it)
       cout<< (*it)<<"   ";

  cout <<"\nremoving from front ... "<<endl;
        dq.pop_front();
        cout << "front : "<<dq.front()<<endl;
        cout << "back : "<<dq.back()<<endl;

cout<<"\nelements in the queue :\n";
      for(auto it = dq.begin(); it != dq.end(); ++it)
            cout<< (*it)<<"   ";

cout <<"\nremoving from rear ... "<<endl;
  dq.pop_back();
        cout << "front : "<<dq.front()<<endl;
        cout << "back : "<<dq.back()<<endl;


cout<<"\nelements in the queue :\n";
      for(auto it = dq.begin(); it != dq.end(); ++it)
          cout<< (*it)<<"   ";
      cout<<endl;

    if (! dq.empty() )
     cout <<"size of queue = "<< dq.size()<<endl;

  system("pause");
return 0;
}
```

Output

```
queue front = 10
queue back = 50
size of queue = 5

elements in the queue :
10   20   30   40   50
removing from front ...
front : 20
back : 50

elements in the queue :
20   30   40   50
removing from rear ...
front : 20
back : 40

elements in the queue :
20   30   40
size of queue = 3
```

Palindrome Checker using Deque

Objective

- To implement a Double-Ended Queue (Deque) data structure using C++ and the STL (std::deque).
- To understand the efficiency of bidirectional data access in verifying symmetrical patterns.
- To implement a symmetry-verification algorithm using a Deque.

Implementation

- **Input**: A string (word or phrase).
- **Output**: A boolean result indicating whether the input is a palindrome.
- **Logic**: A palindrome reads the same forwards and backwards. By using a deque, we can compare the "front" and the "back" of the string simultaneously. If at any point the characters at both ends do not match, the symmetry is broken, and it is not a palindrome.

Algorithm

1. **Start.**
2. **Initialize** an empty deque D of type Character.
3. **Input** a string S.
4. **Phase 1 (Preprocessing):** Iterate through each character C in string S:
    - If C is alphanumeric (letter or number):
        - Convert C to lowercase.
        - **Push** C onto the back of deque D.
5. **Phase 2 (Comparison):** While the size of deque D is greater than 1:
    - Get the front element F and the back element B.

- **If** F is not equal to B:
  - Display "Not a Palindrome" and go to **Step 7**.
- **Else**:
  - **Pop** the front element from D.
  - **Pop** the back element from D.

6. **Display** "It is a Palindrome."
7. **End.**

Code:

```
#include <iostream>
#include <string>
#include <deque>
#include <cctype> // for isalnum(ch)
using namespace std;

bool isPalindrome(const string& text)
{
    deque<char> charDeque;

     cout<<"After preprocessing text is:  ";
     for (char ch : text)
      {
        // Only consider alphanumeric characters
      //to filter spaces, commas, exclamation marks, etc. i.e. ignore punctuation
      //If the function returns true, we keep the character; otherwise, we skip it.

   if (isalnum(ch))
       {
          // charDeque.push_back(tolower(ch));
         //or
         char lowch= tolower(ch); //Convert to lowercase for case insensitivity
        cout<< lowch ;
        charDeque.push_back(lowch); // push it to deque
        }
    }
    cout << endl;
```

```
    while (charDeque.size() > 1)
       {
        if (charDeque.front() != charDeque.back())
          {
             return false;
        }
        charDeque.pop_front();
        charDeque.pop_back();
     }

    return true;
}

int main()
{
    string input;

    cout << "=== Palindrome Checker (Using std::deque) ===" << endl;
    cout << "Enter a word or phrase: ";
    getline(cin, input);
   //or
     //cin>> input;
    if (isPalindrome(input))
        cout << "Result: Yes, it's a palindrome!" << endl;
     else
        cout << "Result: No, it's not a palindrome." << endl;

     system("pause");
    return 0;
}
```

Sample outputs:

```
=== Palindrome Checker (Using std::deque) ===
Enter a word or phrase: eye
After preprocessing text is:  eye
Result: Yes, it's a palindrome!
```

```
=== Palindrome Checker (Using std::deque) ===
Enter a word or phrase: Wow!
After preprocessing text is:  wow
Result: Yes, it's a palindrome!
```

```
=== Palindrome Checker (Using std::deque) ===
Enter a word or phrase: Race car
After preprocessing text is:  racecar
Result: Yes, it's a palindrome!
```

```
=== Palindrome Checker (Using std::deque) ===
Enter a word or phrase: Hello
After preprocessing text is:  hello
Result: No, it's not a palindrome.
```

Printer Buffer Simulation using Queue

Objectives

- To implement a FIFO data structure using std::queue.
- To simulate real-time task processing where data is handled in the order of arrival.
- To manage custom data types (struct) within an STL container.

Implementation

- **Input**: A series of tasks, each with a name/ID and a simulated "pages to print" (processing time).
- **Output**: A step-by-step log of the printer starting, processing, and finishing each job.
- **Logic**: Jobs enter the queue at the back and are served from the front. The program won't start Job 2 until Job 1 is completely "popped" from the queue.

Algorithm

1. **Start.**
2. **Define a Structure** PrintJob containing id, name, and processingTime.
3. **Initialize** a std::queue of type PrintJob.
4. **Enqueue (Push):** Add several tasks into the back of the queue.
5. **Process (Loop):** While the queue is **not empty**:
    - **Peek:** Access the data at the front() of the queue.
    - **Execute:** Print the document details and "wait" (simulate work) based on the processingTime.
    - **Dequeue (Pop):** Remove the task from the front once finished.
6. **Display** a message when the queue is empty.
7. **End.**

Code:

```
#include <iostream>
#include <string>
#include <queue>
#include <windows.h> // Required for Sleep()

using namespace std;

// Create a struct Task
struct PrintJob
{
    int id;
    string documentName;
    int pages; // This represents the "Processing Time"
};

int main()
{
    // Initialize an empty queue of type PrintJob
    queue<PrintJob> printerBuffer;

    cout << "=== Printer Buffer System (FIFO) ===\n" << endl;

     PrintJob job[3] = {{101, "Lab_Report.pdf", 3}, {102,
     "Vacation_Photo.jpg", 1},{103, "Lecture_Draft.docx", 4}}  ;

 // Adding tasks to the queue
// (Simulating Arrival)
    printerBuffer.push(job[0]);
    printerBuffer.push(job[1]);
    printerBuffer.push(job[2]);
```

OR:

```
  PrintJob job[3];
  for(int i=0; i< 3; i++)
    {
     cout<<"Enter job ("<< i+1 <<") details\n";
       cout <<"ID : " ; cin>> job[i].id;
       cout <<"\nName : "; cin>> job[i].documentName;
       cout <<"\nNo. of pages : "; cin>> job[i].pages;
    }
//--------------
  cout<<"adding Jobs  to Printer Buffer:\n";
    for(int i=0; i< 3; i++)
    printerBuffer.push(job[i]);
```

```
    cout<<"Jobs added to Printer Buffer:\n";
       for(int i = 0; i<3 ; i++)
         {
              cout<<"Job no. "<< i+1 <<" :\n"<<"   id :" <<job[i].id
                   <<"\n   name :" <<job[i].documentName
                   <<"\n   pages :" <<job[i].pages<<endl;
         }

   cout << "\nAdded 3 jobs to the queue. Starting printer..." << endl;

    // Processing (The FIFO Loop) & Simulating "Processing Time"

    while(!printerBuffer.empty())
    {
       PrintJob current = printerBuffer.front();
      cout << "\nPrinting : " << current.documentName << "..." << endl;

       // Windows-specific sleep (milliseconds)
       Sleep(current.pages * 1000);

      cout << "Finished after " << current.pages << " seconds." << endl;
      cout << "Job Finished! Document removed from buffer.\n" << endl;
       printerBuffer.pop(); // Pop the element after completion
    }

   // End of printing
   cout << "All print jobs completed. Printer Idle." << endl;
    system("pause");
   return 0;
}
```

Output:

```
=== Printer Buffer System (FIFO) ===

Jobs added to Printer Buffer:
Job no. 1 :
   id :101
   name :Lab_Report.pdf
   pages :3
Job no. 2 :
   id :102
   name :Vacation_Photo.jpg
   pages :1
Job no. 3 :
   id :103
   name :Lecture_Draft.docx
   pages :4

Added 3 jobs to the queue. Starting printer...

Printing : Lab_Report.pdf...
Finished after 3 seconds.
Job Finished! Document removed from buffer.


Printing : Vacation_Photo.jpg...
Finished after 1 seconds.
Job Finished! Document removed from buffer.


Printing : Lecture_Draft.docx...
Finished after 4 seconds.
Job Finished! Document removed from buffer.

All print jobs completed. Printer Idle.
```